“Alarm Clock Using Linux System Programming”

A Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY in **DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

Course: Operating Systems and Systems Programming (25CS2104E) | Term-I, 2026–27 | Section 04 | Team 05

By

Student Name	Roll Number
VARAKALA TARUN	2520030526
K. ROHAN	2520030526
P. SHIVASAI	2520030596

Under the Esteemed Guidance of **[Name of Guide]**, Assistant Professor, Department of Computer Science and Engineering

K L (Deemed to be) University

Declaration

The Project Report entitled “Alarm Clock Using Linux System Programming” is a record of bonafide work of VARAKALA TARUN - 2520030526, K. ROHAN - 2520030526, P. SHIVASAI - 2520030596, submitted in partial fulfillment for the award of B.Tech in Computer Science and Engineering to K L University. The results embodied in this report have not been copied from any other department, university or institute.

VARAKALA TARUN (2520030526) | K. ROHAN (2520030526) | P. SHIVASAI (2520030596)

Certificate

This is to certify that the mini project based report entitled “Alarm Clock Using Linux System Programming” is a bonafide work done and submitted by VARAKALA TARUN - 2520030526, K. ROHAN - 2520030526 and P. SHIVASAI - 2520030596, in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2026-2027.

Signature of the Guide | Signature of the Course Coordinator | Signature of the HOD

Acknowledgement

The success of this project would not have been possible without the timely help and guidance of many people. We express our sincere thanks to all those who assisted us, in one way or another, in completing this project.

Our greatest appreciation goes to our Course Coordinator and our guide, Department of Computer Science, whose support, encouragement and guidance made this project possible. We express our gratitude to the Head of the Department of Computer Science Engineering for providing the facilities and the environment in which we could complete this project-based lab.

We thank all the teaching and non-teaching staff who assisted us directly or indirectly. Finally, we thank our parents, friends and classmates for their help and cooperation during our work.

VARAKALA TARUN (2520030526) | K. ROHAN (2520030526) | P. SHIVASAI (2520030596)

Abstract

Most people use an alarm clock every day without ever asking how a computer knows when to wake them. General-purpose alarm applications hide the operating-system mechanisms that make time-based events possible: kernel timers, signal delivery, process scheduling and persistent state. To tackle this, our project focuses on the Design and Development of an Alarm Clock Using Linux System Programming. The aim was to build a lightweight, command-line alarm utility in C that schedules and triggers alarms accurately, while demonstrating how a user-level program cooperates with the Linux kernel.

The application is organised into four cooperating modules. The Alarm Setting module parses a clock time such as 07:30:00 or a duration such as 1h30m, computes the number of seconds to wait, and creates a background child process with fork(). The Signal Handling module installs handlers with sigaction(), blocks signals with sigprocmask(), arms the kernel timer with alarm(), and waits for SIGALRM with sigsuspend(), so the process consumes no CPU while it waits. The Persistence and Logging module stores every alarm in a text database protected by flock() and appends events to a log file. The Command-Line module provides the commands set, in, list, cancel and clear.

The most useful feature is that each alarm lives in its own detached process, so alarms keep running after the command that created them has returned and several alarms can be pending at once. In our measurements an idle waiting alarm used about 1 MB of resident memory and no measurable CPU, and four alarms armed for 2, 5, 8 and 12 seconds each fired at the exact second requested. The program is developed in C, compiled with GCC on Ubuntu with -Wall -Wextra and no warnings, and is verified by 21 automated unit checks plus functional tests. In summary, this project delivers a simple, reliable and educational alarm utility that turns kernel-level primitives into an everyday tool.

Project Team and Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030526	Varakala Tarun	Timer and signal design: alarm(), sigaction(), sigprocmask(), sigsuspend(); alarm child process and fork()/setsid() flow
2520030526	K. Rohan	Command-line interface, time and duration parsing, input validation, Makefile and build
2520030596	P. Shivasai	Persistence and logging with flock(), testing and measurements, screenshots and documentation

Index

- Acknowledgement, Abstract

- Chapter 1 Introduction: 1.1 Background of the Project; 1.2 Problem Statement; 1.3 Scope of the Project; 1.4 Objectives of the Project; 1.5 Importance of the Application; 1.6 Target Users / Audience
- Chapter 2 OS Concepts and Background Study: 2.1 Operating System Concepts Related to the Project; 2.2 Existing Methods / Algorithms; 2.3 Proposed Approach; 2.4 Advantages of the Proposed Approach
- Chapter 3 System Requirements: 3.1 Hardware Requirements; 3.2 Software Requirements; 3.3 Operating System Environment – Ubuntu / WSL; 3.4 Development Tools and Compilers
- Chapter 4 Technology and Tools Used: 4.1 C / C++ Programming; 4.2 GCC Compiler; 4.3 Linux System Calls; 4.4 POSIX Threads / Synchronization; 4.5 Linux Commands and Utilities; 4.6 Other Tools / Libraries Used
- Chapter 5 System Architecture and Design: 5.1 Overall System Architecture; 5.2 System Workflow; 5.3 Flowchart / Data Flow; 5.4 Module Description; 5.5 Input and Output Design
- Chapter 6 Implementation: 6.1 Module-wise Implementation; 6.2 Algorithm / Pseudocode; 6.3 Core C Implementation; 6.4 Linux System Calls / APIs Used; 6.5 Process / Thread Implementation; 6.6 IPC / Synchronization Implementation; 6.7 Memory Management Implementation; 6.8 Optional User Interface Implementation
- Chapter 7 Execution and Demonstration: 7.1 Compilation in Ubuntu; 7.2 Execution Commands; 7.3 Sample Inputs and Outputs; 7.4 Screenshots of Execution; 7.5 Demonstration of OS Concepts
- Chapter 8 Conclusion: 8.1 Summary of the Project; 8.2 Objectives Achieved; 8.3 Results Achieved; 8.4 Skills and Learning Outcomes
- Chapter 9 References: 9.1 Books; 9.2 Research Papers; 9.3 Linux / GNU Documentation; 9.4 Online Resources

CHAPTER 1 INTRODUCTION

1.1 Background of the Project

Time-based behaviour is everywhere in computing. A laptop wakes a reminder at nine in the morning, a server rotates its log files at midnight, and a download retries after thirty seconds. Behind each of these events is the same idea: a program asks the operating system to tell it when a certain amount of time has passed, and then does something

useful in the meantime, which is often nothing at all. An alarm clock is the simplest complete example of this idea, and that makes it an excellent vehicle for learning how a user program and the kernel work together.

Most alarm applications are graphical programs built on large toolkits. They are convenient, but they conceal the mechanism. A learner who clicks “Set alarm” never sees the timer that the kernel arms, the signal that the kernel delivers when it expires, or the process that sleeps in the meantime. The operating-system course teaches these ideas in theory, yet few projects let a student watch all of them cooperate inside a small, readable program.

The operating-system support behind an alarm

A program cannot measure time by itself in any efficient way. The crude method is a busy loop that reads the clock again and again until the target time arrives, which keeps the processor fully occupied for no benefit. The correct method is to ask the kernel for help. Linux offers several timer facilities: alarm() for a single timer with one-second resolution, setitimer() for interval timers, and POSIX timers created with timer_create() for several independent timers with nanosecond resolution. When a timer expires, the kernel sends the process a signal, SIGALRM, which interrupts whatever the process was doing and runs a handler function registered earlier. Meanwhile the process can sleep in the kernel, using no processor time, until the signal arrives.

Combined with fork(), setsid(), sigaction(), sigprocmask(), sigsuspend() and flock(), these facilities provide everything needed to build a complete alarm clock: parse a time, wait without wasting CPU, notify the user, remember the alarm, and let several alarms run independently.

The Alarm Clock project

This project, the Alarm Clock Using Linux System Programming, is a small command-line utility written from scratch in the C language for Linux. The user types a clock time such as 07:30 or a duration such as 25m. The program computes the delay, creates a background process, arms a kernel timer, and sleeps. When the timer fires, the handler wakes the process, which rings the terminal bell, prints a banner with the user’s message, updates the alarm database and writes a log entry. The user can list alarms, cancel a pending alarm or clear finished ones at any time.

File	Lines	Role in the project
include/alarm.h	45	Shared types, constants and function prototypes
src/main.c	72	Command-line front end: set, in, list, cancel, clear, help
src/alarm_set.c	133	Time parsing, delay computation, fork() and the alarm child process, cancel and list
src/signal_handler.c	56	SIGALRM/SIGTERM handlers, race-free waiting and the notification banner
src/logger.c	142	flock-protected alarm database and the event log
tests/test_parse.c	48	21 automated unit checks for parsing and delay computation
tests/accuracy.sh	5	Timing-accuracy experiment with four simultaneous alarms
Makefile	18	Builds the alarm_clock executable with -Wall -Wextra

The core program is under 450 lines. A reader can study the complete source in one sitting and can trace every feature of the alarm clock to a specific Linux system call. That visibility is the main reason the project was built.

Technology snapshot

Item	Detail
Language	C (C11, compiled with GCC 13.3, flags -Wall -Wextra -std=c11 -g -O0)
Platform	Linux (developed and tested on Ubuntu 24.04, kernel 6.x, x86-64)
Key system calls	fork, setsid, alarm, sigaction, sigprocmask, sigsuspend, kill, flock, rename, time
User interface	Command line (alarm_clock) with a terminal bell and text banner

Item	Detail
Persistence	Plain-text database ~/.alarm_clock.db and log ~/.alarm_clock.log
Build result	Compiles with no warnings; 21 of 21 unit checks pass

1.2 Problem Statement

Conventional alarm applications hide the operating-system mechanisms involved in time-based event handling. Students of operating systems learn the theory of processes, signals, timers and system calls, but they seldom see these ideas working together in one practical utility. An alarm clock is the ideal such utility, yet the ones in everyday use reveal nothing of how they work.

Difficulties faced by learners

1. **The mechanism is hidden.** A GUI alarm shows that a sound played at 7:30, but not how the program knew when to play it. Learners do not see the timer, the signal or the sleeping process.
2. **Busy waiting is tempting.** A beginner's first solution is a loop that polls the clock. It works, but it burns CPU and teaches the wrong lesson about how an operating system should be used.
3. **Signals are abstract.** Concepts such as signal delivery, handlers, masks and the race between checking a flag and going to sleep remain theoretical when there is no small program that depends on them.
4. **Shell tools are blunt.** Combinations such as sleep followed by a notification command, or cron and at, hide process control and give little control over cancelling or listing pending alarms.
5. **State is lost.** A simple alarm program forgets its alarms when it exits, and several instances that write to one file can corrupt it unless they synchronise.

Gap analysis

What learners need	What common tools provide	Gap
See the timer, signal and handler of an alarm	Hidden inside a GUI toolkit	No visibility

What learners need	What common tools provide	Gap
A small, readable code base	Large frameworks	Cannot be studied
Several independent alarms	One alarm per app or manual sleep commands	Poor control
Cancel and list alarms	Not available in sleep-based scripts	Missing management
Zero CPU while waiting	Busy-wait loops in naive programs	Wasteful
Safe shared state	Ad-hoc files without locking	Risk of corruption

Problem definition

The problem addressed by this project is therefore: to design and implement a small, readable alarm clock for Linux that exposes the operating-system mechanisms behind time-based events – timers, signals, signal masks, process creation, background processes, file locking and persistent state – and that lets a user schedule, list and cancel alarms from a simple command line.

Success criteria

Criterion	How it is checked
A clock-time alarm fires at the requested second	Banner and log entry appear at the target time
A duration alarm fires after the requested delay	Alarms for 2, 5, 8 and 12 seconds fire exactly on time
The command returns immediately	Prompt is available right after “Alarm #n set for …”
Several alarms run at once	list shows multiple PENDING entries with different PIDs
A pending alarm can be cancelled	Status becomes CANCELLED and the process exits
The waiting process uses no CPU	ps shows 0.0 %CPU and state S in sigsuspend
Wrong usage gives a clear message	Messages such as “invalid time ‘25:61’” instead of a crash

1.3 Scope of the Project

The scope describes what the Alarm Clock does and, equally important, what it does not do. It was kept narrow on purpose so that every feature could be implemented completely and explained clearly.

Features within the scope

Feature	What it does	Limit
Set by clock time	set HH:MM[:SS] schedules the next occurrence of that time	Next 24 hours; seconds resolution
Set by duration	in 30s, 5m, 1h30m schedules a relative alarm	1 second to 24 hours
Custom message	Any text after the time is shown when the alarm rings	127 characters

Feature	What it does	Limit
Background operation	fork() and setsid() put each alarm in a detached process	One process per alarm
Notification	Three terminal bells and a framed banner	Console only
List	Shows id, PID, time, status and message	Up to 64 alarms
Cancel	Sends SIGTERM to the waiting process and marks it CANCELLED	Pending alarms only
Clear	Removes finished alarms from the database	—
Persistence and log	Text database and event log in the home directory	Plain text

Features outside the scope

Not included	Reason
Graphical user interface	The project is about system calls, not toolkits
Alarms that survive a reboot	Would need an init system or cron integration; alarm processes end when the machine stops
Recurring alarms (daily, weekly)	Adds calendar logic that does not teach new OS concepts
Sound files and desktop notifications	Depend on external packages; the terminal bell is enough to demonstrate the signal path
Sub-second precision	alarm() has one-second resolution; POSIX timers would be needed
Snooze, alarm editing	Possible future work (Chapter 8)

Assumptions and constraints

- The user runs a POSIX-compliant Linux system such as Ubuntu 20.04 or later, with a writable home directory.
- The system clock is correct; the program trusts the value returned by time().

- The terminal that started an alarm is still open when it rings; otherwise the banner is lost but the log and database still record the event.
- Alarm times are interpreted in the local time zone. A clock time that has already passed today is scheduled for tomorrow.

1.4 Objectives of the Project

The objectives of this project are to design, build, test and document a command-line alarm clock that demonstrates practical operating-system concepts.

1. To design and implement a command-line alarm clock using Linux and POSIX system programming.
2. To schedule alarms and generate a notification at exactly the specified time.
3. To demonstrate signal handling, kernel timers, time management and process-related OS concepts.
4. To keep the waiting process free of busy waiting so that it uses no CPU while idle.
5. To support several independent alarms that survive the command that created them.
6. To protect shared state with file locking and atomic file replacement.
7. To develop, test, document and demonstrate the working solution on Ubuntu/Linux.

Measurable objectives

No.	Objective	Measure	Result
1	Accurate scheduling	Fire time equals target time at one-second resolution	Achieved: 4 of 4 alarms exact
2	No busy waiting	CPU use while idle	0.0 %CPU, state S (sleeping)
3	Small footprint	Resident memory of an idle alarm	1064 kB
4	Input validation	Invalid inputs rejected with a message	6 of 6 invalid inputs rejected
5	Code quality	Compiler warnings under -Wall -Wextra	0 warnings
6	Verification	Automated unit checks	21 of 21 pass

1.5 Importance of the Application

The importance of the application lies in what it teaches and in what it demonstrates, rather than in competing with commercial alarm apps.

- **Educational value.** The project joins five of the most important ideas of an operating-systems course in one program: processes, signals, timers, system calls and synchronisation. Each idea is visible in a few lines of code.
- **Correctness lessons.** The waiting routine uses the sigprocmask() and sigsuspend() pair to avoid the classic lost-wakeup race. Understanding why that pair is needed is a lesson that applies to all concurrent programming.
- **Efficiency.** The program shows the difference between asking the kernel to wake a process and polling for the time. The measured idle cost of about 1 MB and zero CPU is the practical proof.
- **Headless and embedded use.** Because the program needs no graphics, it runs on servers, containers, SSH sessions and small embedded boards where a GUI alarm is impossible.
- **Foundation for extension.** The module structure makes it easy to add recurring alarms, a daemon, a sound player or a graphical front end later without changing the timer and signal core.

1.6 Target Users / Audience

User group	How they benefit
Students of operating systems and systems programming	A small, complete example of timers, signals and process control that can be read, run and modified
Instructors	A ready demonstration for lectures and lab sessions on signals and kernel timers
Developers and system administrators	A lightweight reminder tool for terminal sessions on servers without a desktop
Embedded and headless-system users	An alarm utility with no GUI dependencies and a footprint of about 1 MB
Future project teams	A base on which to build schedulers, daemons or notification front ends

CHAPTER 2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The alarm clock is built from seven operating-system ideas. Each is introduced here in the order in which a request travels through the program, from the moment the user presses Enter to the moment the banner appears.

Processes and fork()

A process is a running program together with its address space, open files and kernel bookkeeping. The fork() system call creates a new process that is an almost exact copy of the caller. It returns twice: it returns the new process's identifier (PID) to the parent and zero to the child. Linux implements the copy lazily with copy-on-write, so pages are shared until one process writes to them, which makes fork() cheap. In this project fork() separates two roles. The parent is short-lived: it parses the command, records the alarm and returns the prompt to the user. The child is long-lived: it waits for the alarm time. Because the two are separate processes, the command returns immediately while the alarm keeps waiting.

An important property of the design is that an alarm timer set with alarm() is not inherited by a child created with fork(), and the child starts with no pending signals. The child therefore arms its own timer after the fork, which is exactly what the program does.

Sessions, process groups and setsid()

Every process belongs to a process group and a session. A session may have a controlling terminal, and when that terminal is closed the kernel sends SIGHUP to the foreground process group. A background alarm that belongs to the user's login session would be exposed to this signal. The setsid() call makes the caller the leader of a new session with no controlling terminal, so the alarm child is detached from the terminal job-control machinery. When the parent exits, the orphaned child is adopted by the init process, which collects its exit status when it finishes, so no zombie process is left behind.

Signals

A signal is a small asynchronous notification that the kernel delivers to a process. Each signal has a number and a default action. SIGALRM (14 on Linux x86) is delivered when a timer set with alarm() or setitimer() expires; its default action is to terminate the process, so a program that arms a timer must install a handler first. SIGTERM (15) is the polite

request to terminate that the cancel command uses, and SIGINT (2) is what Ctrl+C sends. A process may handle, ignore or block a signal, with the exception of SIGKILL and SIGSTOP.

Signal	Number	Sent when	Use in the project
SIGALRM	14	alarm() timer expires	Wakes the waiting alarm process
SIGTERM	15	kill() with default signal	cancel command ends a pending alarm
SIGINT	2	Ctrl+C at the terminal	Ends a waiting alarm cleanly
SIGKILL	9	kill -9	Cannot be caught; ends the process at once

A signal handler runs asynchronously, possibly between any two instructions of the main program. It may therefore call only async-signal-safe functions, and it may share data with the main program only through variables of type volatile sig_atomic_t. This is why the handlers in this project do nothing except set a flag; all real work, such as printing the banner and writing the log, is done by the main flow after the handler returns.

Kernel timers and the alarm() system call

The call alarm(n) asks the kernel to send SIGALRM to the calling process after n real seconds. A process has only one such timer: a second call replaces the first, and alarm(0) cancels it. Inside Linux, alarm() is implemented on top of the real-time interval timer ITIMER_REAL, which is driven by a high-resolution kernel timer. When the timer expires, the kernel marks SIGALRM as pending for the process; the signal is delivered the next time the process is about to return to user mode. If the process is asleep in sigsuspend(), the pending signal wakes it. The resolution of alarm() is one second, which is exactly what a human alarm clock needs.

Time management

The call time() returns the number of seconds since the Unix epoch, 1 January 1970 UTC. The functions localtime_r() and mktime() convert between this count and the broken-down calendar form struct tm, taking the time zone and daylight-saving rules into account. To set an alarm for 07:30, the program fills a struct tm with today’s date and the

requested hour and minute, converts it with mktime(), and subtracts the current time. If the result is not in the future, one day is added. The reentrant variant localtime_r() is used instead of localtime() because it does not rely on a hidden static buffer.

Signal masks and the lost-wakeup race

Each process has a signal mask, the set of signals currently blocked from delivery. Blocked signals stay pending until they are unblocked. The mask exists to solve a classic race. Suppose the program tests a flag that says whether the alarm has fired and, if not, calls pause() to sleep. If SIGALRM arrives after the test but before pause() begins, the handler sets the flag, but the process then sleeps forever because the signal that would wake it has already been consumed.

The cure is to block SIGALRM with sigprocmask() before testing the flag, and then to call sigsuspend(), which atomically restores the old mask and puts the process to sleep. A signal that arrives while the mask is blocked remains pending and is delivered the instant sigsuspend() unblocks it, so the wake-up cannot be lost. This is the single most instructive piece of the project and is implemented in wait_for_signal().

File locking and atomic update

The alarm database is shared by several processes: the command that adds an alarm, each waiting child that marks its alarm DONE, and the cancel, list and clear commands. If two of them rewrite the file at once, one update can overwrite the other. The project uses flock(), an advisory lock on an open file, to give one process at a time exclusive access, and it writes the new content to a temporary file that it then rename()s over the old one. On a POSIX file system rename() is atomic, so a reader never sees a half-written database.

Concept-to-code map

OS concept	System call or API	Where used
Process creation	fork()	start_alarm() in alarm_set.c
Session detach	setsid()	alarm_child() in alarm_set.c
Kernel timer	alarm()	alarm_child() in alarm_set.c
Signal handling	sigaction()	install_handlers() in signal_handler.c
Signal masking	sigprocmask()	wait_for_signal() in signal_handler.c
Race-free sleep	sigsuspend()	wait_for_signal() in signal_handler.c

OS concept	System call or API	Where used
Signalling another process	kill()	cancel_alarm(), list_alarms() in alarm_set.c
Time of day	time(), localtime_r(), mktime()	main.c, alarm_set.c, logger.c
Advisory locking	flock()	lock_db() in logger.c
Atomic file replace	rename()	save_nolock() in logger.c
Process termination	_exit()	alarm_child() in alarm_set.c

2.2 Existing Methods / Algorithms

Several methods exist for waiting until a particular time. Each makes different trade-offs, and understanding them explains the choices made in this project.

Busy waiting

The simplest algorithm reads the clock in a loop and exits when the target time is reached. It is easy to write but keeps one processor core at full load for the entire wait. On a battery-powered machine this drains the battery, and on a shared server it steals time from other work. It is the method that the project is designed to avoid.

sleep() and nanosleep()

The sleep() call suspends the process for a number of seconds, and nanosleep() does the same with nanosecond precision. They do not use the processor while waiting, but the process can do nothing else, cannot be told to stop except by a signal, and a shell command such as sleep 600 followed by a notification gives no way to list or cancel pending alarms. On some systems sleep() is itself implemented with SIGALRM, so mixing it with alarm() in the same process is unsafe.

cron and at

The cron daemon runs commands at times given in a table, and at runs a command once at a given time. Both are reliable and survive logging out, but their resolution is one minute, they require a running daemon and its configuration, and they run jobs without a terminal, so showing a message to the user needs extra tools. They are excellent for system maintenance and poor for an interactive alarm with second-level precision.

systemd timers

Modern distributions offer timer units managed by systemd. They support calendar expressions, persistence across reboots and precise accuracy settings. They need unit files and root or user-manager privileges, and they hide the signal and timer mechanics completely.

Event-loop timeouts: select(), poll(), timerfd

A program that already waits on file descriptors can pass a timeout to select() or poll(), or create a timer represented as a file descriptor with timerfd_create(). This integrates time with input handling and is the preferred method for servers. It is a slightly more advanced design than needed for a single-purpose alarm and conceals the classic signal path.

POSIX timers

The timer_create() family offers several independent timers per process, nanosecond resolution, a choice of clocks and a choice between signal and thread notification. They are the most flexible kernel timers. The price is more setup code and a link against the real-time library on older systems.

Method	Resolution	CPU while waiting	Cancel / list	Shows the OS mechanism
Busy wait	Very fine	100 % of a core	Manual	No
sleep / nanosleep	1 s / 1 ns	None	Hard	Partly
cron / at	1 minute	None	Yes, via tools	No
systemd timer	Configurable	None	Yes, via systemctl	No
select / poll / timerfd	ms to ns	None	In program	Partly
POSIX timer	1 ns	None	In program	Yes, but more setup
alarm() + SIGALRM (this project)	**1 s**	**None**	**Yes (cancel, list)**	**Yes, fully visible**

2.3 Proposed Approach

The proposed system uses alarm() and SIGALRM as its core, because that pair exposes the signal-and-timer mechanism most directly and its one-second resolution fits an alarm clock. Because alarm() allows only one timer per process, the design gives each alarm its own process. This choice is deliberate: it turns the process model itself into the mechanism for multiple alarms, and a process that is waiting costs the system almost nothing.

Design decisions

Decision	Reason
One detached child per alarm	alarm() has one timer per process; separate processes give independent alarms and survive the shell command
Arm the timer in the child, after fork()	Timers are not inherited by children; arming after the fork is correct and avoids a race
Handlers only set flags	Async-signal safety; heavy work is done in normal code
sigprocmask() + sigsuspend()	Removes the lost-wakeup race between testing the flag and sleeping
sa_flags = 0 (no SA_RESTART)	A restarted sleep would not return when the signal arrives
Text database with flock() and rename()	Human-readable state shared safely by several processes
Reserve the alarm id before fork()	The id exists in the database before the child can finish, so DONE can always be recorded
Persist the child PID after fork()	cancel needs the PID to send SIGTERM

Outline of the algorithm

1. Read the command-line arguments and validate them.
2. Convert the requested time or duration into an absolute target time.
3. Store a PENDING record in the database and obtain its id.
4. Call fork(). In the parent, save the child's PID, print the confirmation and exit.

5. In the child, call setsid(), install the handlers, compute the delay and call alarm(delay).
6. Block the signals, then loop in sigsuspend() until SIGALRM or SIGTERM arrives.
7. If SIGALRM arrived, ring the bell, print the banner, set the status DONE and log the event.
8. If SIGTERM or SIGINT arrived, log the early termination.
9. Exit the child with _exit(0).

2.4 Advantages of the Proposed Approach

- **No busy waiting.** The waiting process is asleep in the kernel and consumes no CPU, as the measurement of 0.0 %CPU in Chapter 7 confirms.
- **Small and transparent.** About 450 lines of C, four modules and no external libraries. Every behaviour maps to a named system call.
- **Several alarms at once.** Each alarm is an independent process, so alarms never interfere with each other and can be cancelled individually.
- **Command returns immediately.** The user can keep working in the same terminal after setting an alarm.
- **Race-free.** The mask-and-suspend sequence cannot lose a wake-up, and the id is reserved before the fork.
- **Safe shared state.** flock() serialises database access and rename() makes every update atomic.
- **Portable.** The program uses only POSIX interfaces plus flock(), which is available on Linux, macOS and the BSD family, so it builds on any POSIX-compliant Linux distribution.
- **Observable.** Because each alarm is a real process, tools such as ps, top and /proc show its state, memory and signal masks, which turns the program into a live teaching aid.

CHAPTER 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The alarm clock is deliberately light and needs only modest hardware. The figures below are minimum values for building and running the project, and the recommended values give a comfortable development experience.

Component	Minimum	Recommended
Processor	Any 64-bit x86 or ARM CPU, 1 GHz	Dual-core 2 GHz or better
Memory (RAM)	512 MB	4 GB or more
Disk space	50 MB for the source and build	1 GB free for tools and logs
Display	Text terminal	Terminal with colour and Unicode support
Audio	Not required	Speaker for the terminal bell
Input	Keyboard	Keyboard

The memory footprint of the program itself is tiny. A waiting alarm process measured 1064 kB of resident memory and 2700 kB of virtual memory. Sixty-four alarms, the configured maximum, would therefore need roughly 70 MB of resident memory in the worst case, which any modern machine can supply.

The terminal bell depends on the terminal emulator: some beep through the sound card, some flash the screen and some ignore the bell character. For this reason the banner is always printed in addition to the bell.

3.2 Software Requirements

Software	Version	Purpose
Linux operating system	Ubuntu 20.04 or later, or any POSIX-compliant distribution	Provides the kernel timers, signals and processes
GCC	9 or later (13.3 used)	Compiles the C11 source
GNU Make	4.x	Drives the build from the Makefile
C standard library (glibc)	2.31 or later	Supplies the system-call wrappers

Software	Version	Purpose
Header files	unistd.h, signal.h, time.h, sys/types.h, sys/file.h, stdio.h, stdlib.h, string.h, errno.h, ctype.h, fcntl.h, stdarg.h	Declarations of the APIs used
Text editor or IDE	VS Code, vim or nano	Writing and reading the source
Debug and inspection tools	gdb, strace, ps, /proc	Debugging and observing the running alarm
Documentation tools	Any word processor	Preparing this report

No third-party library is linked. The program depends only on the C standard library and the kernel interfaces. The optional notify-send command, from the libnotify package, is mentioned as a future extension but is not required.

3.3 Operating System Environment – Ubuntu / WSL

The project was developed and tested on Ubuntu 24.04 LTS running a 6.x Linux kernel on x86-64. Ubuntu was chosen because it is the distribution recommended for the course, ships GCC and Make in its build-essential package, and provides the full set of POSIX interfaces that the project relies on.

Running on Windows with WSL

Students who use Windows can run the project under the Windows Subsystem for Linux. WSL 2 runs a real Linux kernel in a lightweight virtual machine, so fork(), alarm(), sigaction() and flock() behave exactly as on a native installation. The steps are: install WSL with an Ubuntu distribution, open the Ubuntu terminal, install the build tools, and keep the project files inside the Linux home directory rather than under the mounted Windows drives, because the database relies on flock() and rename() semantics that the Linux file system provides most faithfully.

Environment check

Check	Command	Expected result
Kernel and architecture	uname -a	A Linux kernel on x86_64 or aarch64

Check	Command	Expected result
Compiler present	gcc --version	GCC 9 or later
Make present	make --version	GNU Make 4.x
Home directory writable	touch ~/.t && rm ~/.t	No error
Installing tools if missing	sudo apt install build-essential	gcc, make and libc headers installed

3.4 Development Tools and Compilers

GCC

The GNU Compiler Collection translates the C source into machine code. The project is compiled with the flags shown below, which turn on a broad set of warnings and keep debugging information.

Flag	Meaning
-Wall -Wextra	Enable the common and extra warning groups so that suspicious code is reported
-std=c11	Use the C11 language standard
-D_GNU_SOURCE	Make POSIX and GNU declarations such as setsid() and flock() visible under strict C11 mode
-g	Include debugging information for gdb
-O0	Disable optimisation so that the code runs exactly as written and is easy to step through
-Iinclude	Search the include directory for alarm.h

GNU Make

The Makefile lists the four source files, compiles each into an object file with a pattern rule that depends on alarm.h so that a change to the header rebuilds everything, and links them into the alarm_clock executable. The targets are all and clean.

Debugging and observation tools

- **gdb** allows single-stepping through the program, setting breakpoints in the signal handlers and inspecting the pending-signal state. When debugging, the command handle SIGALRM nostop pass stops gdb from halting on every timer signal.
- **strace** records every system call a program makes and is the best way to see fork, alarm, rt_sigaction, rt_sigprocmask and rt_sigsuspend in order. It is listed as a recommended tool; it was not installed on our test machine, so the system-call sequence in Chapter 7 is described from the source and from /proc observations.
- **ps and /proc** show the process state, the kernel function in which a sleeping process waits, resident memory and the masks of blocked and caught signals.

CHAPTER 4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The project is written in C. C is the native language of Unix and Linux: the kernel, the C library and the system-call interface are all defined in terms of C declarations, so a C program can call every system call directly with no translation layer. This matters for a teaching project, because the line of code that calls alarm(5) is a direct request to the kernel and not a call into a framework that might do something else on the way.

Why C rather than C++

C++ could be used for the same project, since it can call the same system calls and include the same headers. C was chosen for three reasons. First, the signal-handling rules are simplest to state in C: a handler is a plain function and shared data is a volatile sig_atomic_t. Second, C gives complete control over memory layout and has no hidden constructors, destructors or exceptions that might run inside a signal handler. Third, C is the language of the course examples, so the code style stays consistent with them.

C features used in the project

Feature	Where it appears	Purpose
struct and typedef	alarm_rec_t in alarm.h	One record holding id, pid, target time, status and message
volatile sig_atomic_t	g_alarm_fired, g_terminate	Flags written by handlers and read by normal code

Feature	Where it appears	Purpose
static functions	path_of(), lock_db(), load_nolock()	Keep helper routines private to a module
Fixed-size arrays	alarm_rec_t all[MAX_ALARM]	No dynamic allocation; predictable memory use
Variadic function	log_event(const char *fmt, ...)	printf-style logging with va_list
Header guards	#ifndef ALARM_H	Allow the header to be included safely more than once
Bounded string functions	strncpy, strncat, snprintf	Prevent buffer overflows
Function pointers (via the API)	sa_handler in struct sigaction	Register the handler functions with the kernel

Coding conventions

The code follows a small set of conventions. Every function has a single clear job. Error returns are checked wherever a failure would matter. Buffers are sized with named constants (MAX_MSG, MAX_ALARM) and all string copies are bounded. Handlers are three lines long. Names describe their action: parse_time_spec, compute_delay, start_alarm, wait_for_signal.

4.2 GCC Compiler

GCC turns the C source into an executable program in four stages, which the gcc command runs one after another.

Stage	Tool	Input	Output	What happens
1. Preprocessing	cpp	main.c	Expanded source	Includes headers, expands macros such as MAX_MSG
2. Compilation	cc1	Expanded source	Assembly	Translates C into assembly for the target CPU
3. Assembly	as	Assembly	main.o	Produces machine code in an object file

Stage	Tool	Input	Output	What happens
4. Linking	ld	Four .o files plus libc	alarm_clock	Resolves function references and creates the executable

The Makefile compiles each of the four source files separately and links them. The build command output is shown in Chapter 7. The warning options -Wall and -Wextra report suspicious code such as unused parameters, so the signal handlers cast their unused argument to void, which documents that it is deliberately ignored. The final build produces no warnings.

The flag -D_GNU_SOURCE deserves a note. When a program is compiled with -std=c11 in strict mode, the compiler hides declarations that are not part of the C standard, including POSIX functions such as setsid() and the BSD-derived flock(). Defining _GNU_SOURCE before the headers are read makes those declarations visible, and it is the standard way to write portable Linux system-programming code under a strict language mode.

4.3 Linux System Calls

A system call is the controlled entry into the kernel. A user program runs with restricted privileges and cannot touch hardware or other processes directly; to do anything of that kind it executes a special instruction that transfers control to the kernel, which checks the request, performs it and returns. The C library wraps each system call in an ordinary function, so alarm(5) in the source becomes a call to a glibc wrapper that loads the system-call number into a register, traps into the kernel and returns the result or sets errno on failure.

System calls and library functions used

Call	Header	Purpose in the project	Returns
fork()	unistd.h	Create the alarm child process	PID to parent, 0 to child, -1 on error
setsid()	unistd.h	Detach the child from the terminal session	New session id, -1 on error
alarm(n)	unistd.h	Ask the kernel for SIGALRM in n seconds	Seconds left on a previous alarm

Call	Header	Purpose in the project	Returns
sigaction()	signal.h	Install the SIGALRM, SIGTERM and SIGINT handlers	0 or -1
sigprocmask()	signal.h	Block the signals before testing the flags	0 or -1
sigsuspend()	signal.h	Atomically unblock and sleep until a signal arrives	Always -1 with errno EINTR
sigemptyset(), sigaddset()	signal.h	Build the signal sets	0 or -1
kill(pid, sig)	signal.h	Send SIGTERM to cancel; test existence with signal 0	0 or -1
time()	time.h	Current time as seconds since the epoch	time_t
localtime_r(), mktime(), strftime()	time.h	Convert between epoch time and calendar time	struct tm, time_t, length
flock()	sys/file.h	Exclusive lock on the database lock file	0 or -1
open(), close()	fcntl.h, unistd.h	Open and release the lock file	fd or -1
rename()	stdio.h	Atomically replace the database	0 or -1
getpid()	unistd.h	Process id shown in log lines	pid_t
usleep()	unistd.h	200 ms pauses between bell characters	0
_exit()	unistd.h	End the child at once without running parent-inherited exit handlers	Does not return

The system-call sequence of one alarm

In order, one alarm in the child makes these kernel requests: fork returns in both processes, setsid creates a new session, rt_sigaction installs each handler, alarm arms the timer, rt_sigprocmask blocks the three signals, rt_sigsuspend sleeps, the kernel delivers SIGALRM and wakes the process, rt_sigprocmask restores the mask, write prints the banner, flock and rename update the database, and exit_group ends the process. Linux names the signal calls with an rt_ prefix because they use the real-time-capable signal interface internally.

Why _exit() instead of exit() in the child? The child is a copy of the parent. If it called exit(), it would run the parent's registered exit handlers and flush copies of the parent's stdio buffers, which could duplicate output. The program therefore flushes stdout before fork() and ends the child with _exit(), which leaves immediately.

4.4 POSIX Threads / Synchronization

POSIX threads (pthreads) let one process run several flows of control that share one address space. Threads are created with pthread_create() and synchronised with mutexes, condition variables and semaphores. The alarm clock does not use threads, and the reasons are worth stating.

Why this project uses processes instead of threads

1. **Signals are process-directed.** A SIGALRM from alarm() is delivered to the process, and in a multithreaded process the kernel may deliver it to any thread that has not blocked it. Controlling which thread wakes would need extra masking code that hides the simple mechanism the project wants to show.
2. **One timer per process.** alarm() gives one timer per process. With one process per alarm, that limit disappears naturally. With threads, several alarms would need POSIX timers or a scheduler thread.
3. **Isolation.** A cancelled or failed alarm process cannot corrupt any other alarm, because they share no memory.
4. **Cancellation.** kill() ends one process cleanly. Ending one thread of many safely is considerably harder.

Synchronization actually needed

Even without threads, concurrent processes share the database file, so synchronisation is still required. The project uses three mechanisms.

Mechanism	Protects	Detail
flock(LOCK_EX) on ~/.alarm_clock.lock	The read-modify-write cycle of the database	Only one process at a time may load, change and save the records
rename() of a temporary file	Readers of the database	A reader sees either the old complete file or the new complete file
Signal mask (sigprocmask + sigsuspend)	The flag-and-sleep sequence	Prevents a lost wake-up between the test and the sleep

A separate lock file is used rather than locking the database itself, because the database is replaced by rename(). A lock held on the old file would not protect the new file that takes its place, so the lock must live on a file that is never replaced.

How the design would change with threads

If a future version were built as a single daemon with one thread per alarm, the shared alarm table would need a pthread_mutex_t, and the waiting threads would use pthread_cond_timedwait() for the timeout. Signals would be blocked in all threads except one dedicated signal thread. This alternative is described only to show the contrast; it is not part of the implementation.

4.5 Linux Commands and Utilities

The commands below were used to build, run, observe and test the alarm clock.

Command	Purpose	Example
make	Build the project from the Makefile	make
gcc	Compile single files or the unit test	gcc -Iinclude -o tests/test_parse tests/test_parse.c src/alarm_set.c src/signal_handler.c src/logger.c
ps	Show the state and memory of the waiting process	ps -o pid,stat,pcpu,rss,vsz,wchan:20,cmd -p PID

Command	Purpose	Example
pgrep, pkill	Find or end alarm processes by name	pgrep -f alarm_clock
kill	Send a signal by hand	kill -TERM PID
cat /proc/PID/status	Read process state, memory and signal masks	grep -E State,VmRSS,SigCgt
date	Read or compute clock times for tests	date -d +4 seconds +%H:%M:%S
sleep	Wait in test scripts for alarms to fire	sleep 5
grep, cat, wc	Inspect the log and database, count lines	cat ~/.alarm_clock.log
uname, gcc --version	Record the environment	uname -a

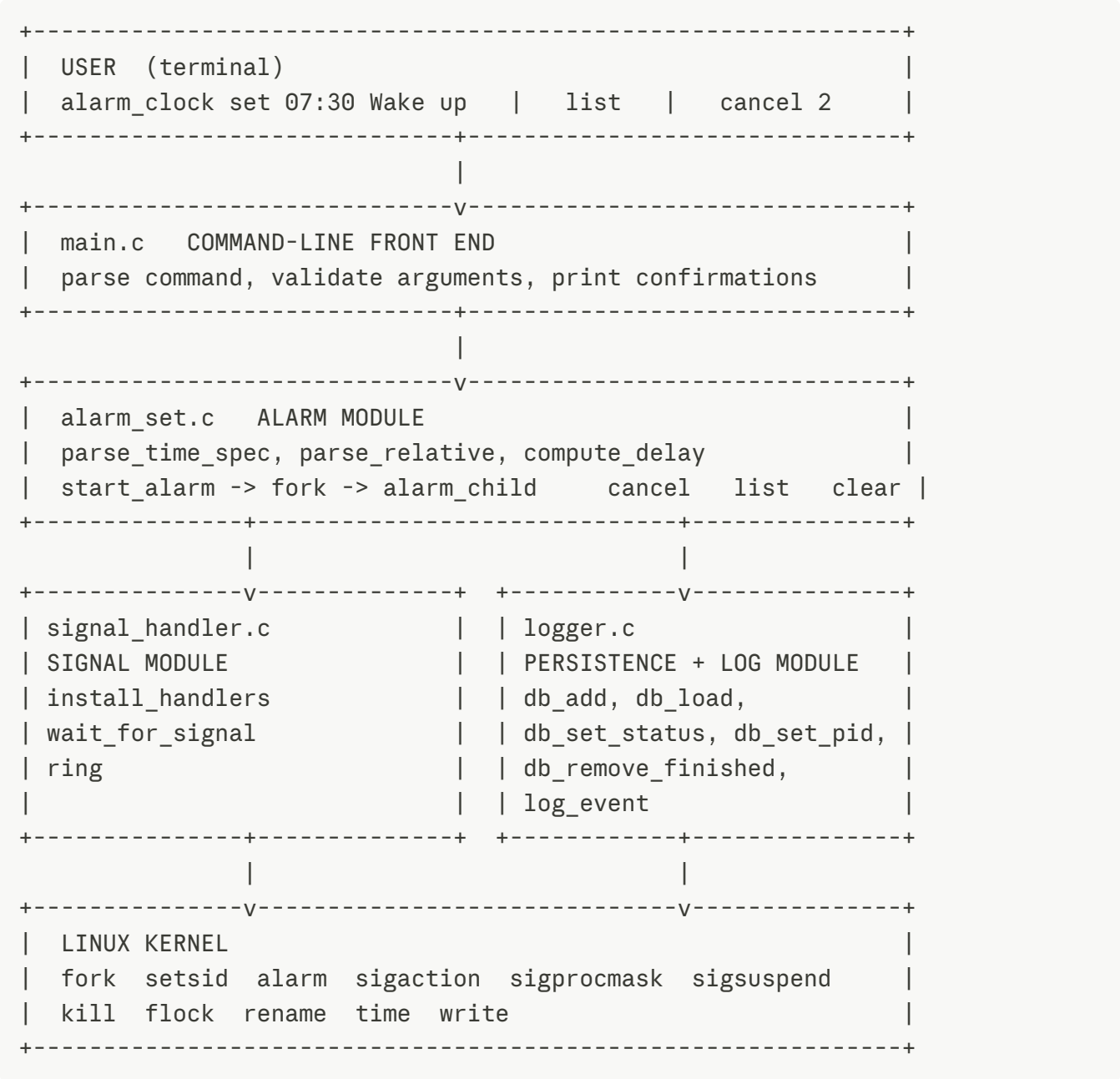
4.6 Other Tools / Libraries Used

- **GNU C Library (glibc).** Supplies the system-call wrappers and the time, string and input/output functions. No other library is linked.
- **POSIX shell scripts.** tests/accuracy.sh arms four alarms and prints the log, and the demonstration commands in Chapter 7 are plain shell.
- **A small unit-test harness.** tests/test_parse.c defines a CHECK macro that prints PASS or FAIL for each condition and returns a non-zero exit status if any check fails. It needs no test framework.
- **The /proc file system.** A virtual file system through which the kernel exposes live process information, used to confirm that the alarm process sleeps in sigsuspend and to read its signal masks.
- **Version control (recommended).** Git is recommended for team work, so that each member's module can be developed on its own branch and merged.
- **Optional notify-send.** Part of libnotify, it can show a desktop pop-up. It is listed in the proposal as a possible extension and is not required by the implementation.

CHAPTER 5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The alarm clock has a layered architecture. At the top, the user interacts with a command-line front end. Beneath it, the alarm module turns a request into a process and a timer. The signal module handles the asynchronous events. At the bottom, the persistence module stores state, and below everything the Linux kernel supplies timers, signals, processes and the file system.



Runtime process architecture

At run time the system is not one program but a small family of processes. Every invocation of alarm_clock set or alarm_clock in creates one short-lived parent and one long-lived child. Other commands such as list and cancel are separate short-lived processes that read and update the shared database.

```
 shell (bash)
   |
   +-- alarm_clock in 5m Tea      (parent, lives about 1 ms)
   |        |
   |        +-- fork() --> child #1  setsid, alarm(300), sigsuspend
(sleeping)
   |
   +-- alarm_clock set 07:30 Wake (parent, lives about 1 ms)
   |        |
   |        +-- fork() --> child #2  setsid, alarm(...), sigsuspend
(sleeping)
   |
   +-- alarm_clock list           (reads the database, exits)
   +-- alarm_clock cancel 1       (kill(child#1, SIGTERM), marks CANCELLED)

  After the parents exit, the children are adopted by init (PID 1).

  Shared state:   ~/.alarm_clock.db    ~/.alarm_clock.log
~/.alarm_clock.lock
```

Design principles

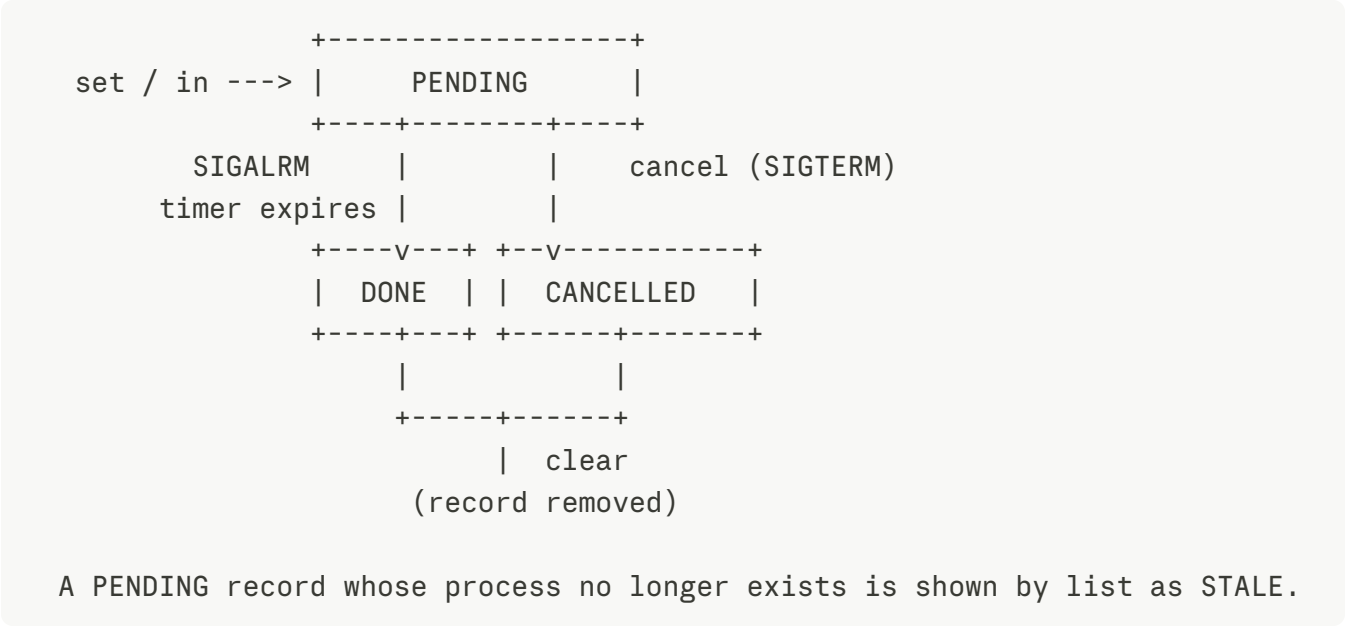
- **Separation of concerns.** Each module has one responsibility and talks to the others only through the prototypes in alarm.h.
- **The kernel does the waiting.** The program never measures time itself while waiting; it asks the kernel and sleeps.
- **Handlers are minimal.** Asynchronous code is kept to the smallest possible size.
- **State is explicit.** Every alarm has a status that moves in one direction: PENDING to DONE or CANCELLED.
- **Fail with a message.** Every invalid input produces a one-line explanation and a non-zero exit status.

5.2 System Workflow

The workflow follows one alarm from request to notification.

Step	Actor	Action	System call or function
1	User	Types the command, for example alarm_clock set 07:30 Wake up	—
2	main()	Reads argv, selects the command	strcmp
3	alarm_set	Converts the time to an absolute target	time, localtime_r, mktime
4	logger	Stores a PENDING record and returns an id	flock, rename
5	alarm_set	Creates the child process	fork
6	Parent	Saves the child PID, prints the confirmation, exits	db_set_pid
7	Child	Detaches from the terminal session	setsid
8	Child	Installs the handlers	sigaction
9	Child	Arms the kernel timer	alarm
10	Child	Blocks the signals and sleeps	sigprocmask, sigsuspend
11	Kernel	Timer expires, delivers SIGALRM	—
12	Handler	Sets g_alarm_fired and returns	—
13	Child	Wakes up, rings the bell, prints the banner	write
14	Child	Sets the status DONE and writes the log entry	flock, rename
15	Child	Exits	_exit

Life cycle of an alarm



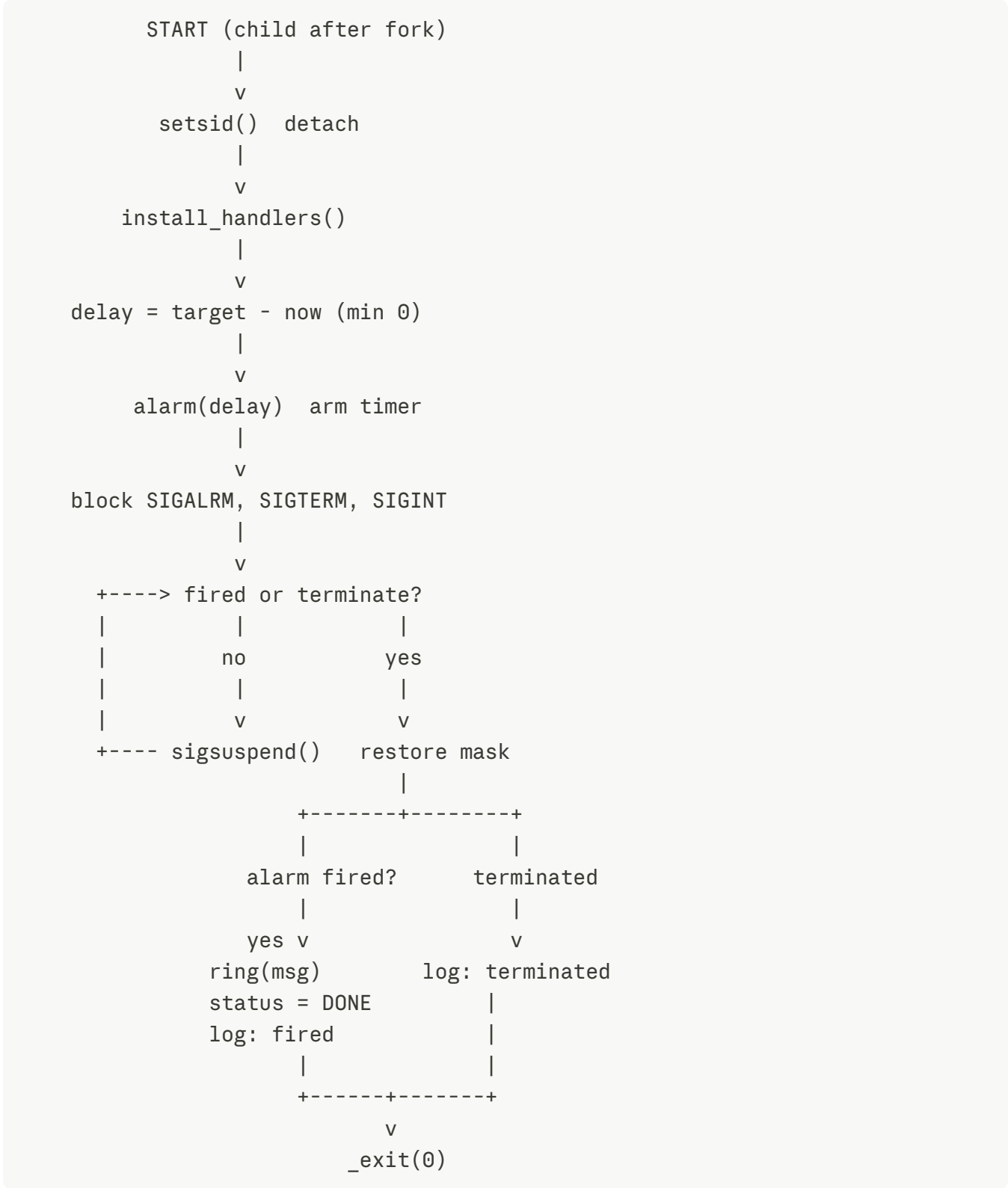
A PENDING record whose process no longer exists is shown by list as STALE.

Timeline of a three-second alarm

Time (s)	Event
0.000	Parent computes target = now + 3 and stores the record
0.001	fork(); parent prints the confirmation and exits
0.002	Child arms alarm(3) and enters sigsuspend()
0.002 to 3.000	Child is asleep; no CPU is used
3.000	Kernel timer expires; SIGALRM is delivered; sigsuspend returns
3.001 to 3.600	Three bell characters, 200 ms apart, then the banner
3.601	Status DONE saved; log line written; child exits

5.3 Flowchart / Data Flow

Flowchart of the alarm child



Flowchart of command parsing

```
argv[1] ?
  |-- none or help  --> print usage
  |-- set  --> parse_time_spec --> invalid? --> error message, exit 1
  |-- in   --> parse_relative  --> invalid? --> error message, exit 1
  |                 |
  |                 v valid
  |           start_alarm(target, message) --> print Alarm #id set ...
  |-- list   --> list_alarms
  |-- cancel --> cancel_alarm(id) --> not found or not pending? --> error
  |-- clear  --> clear_finished
  |-- other  --> error: unknown command, print usage
```

Data flow

Data flow	From	To	Content
D1	User	main.c	Command line text
D2	main.c	alarm_set.c	Validated time string or duration and message
D3	alarm_set.c	logger.c	New alarm record (target time, PENDING, message)
D4	logger.c	alarm_set.c	Assigned alarm id
D5	alarm_set.c	Kernel	alarm(delay) request
D6	Kernel	signal_handler.c	SIGALRM or SIGTERM
D7	signal_handler.c	User terminal	Bell characters and banner
D8	alarm_set.c, signal_handler.c	logger.c	Status changes and log events
D9	logger.c	Files	Database, log and lock file updates

5.4 Module Description

Module 1: Command-line front end (main.c, 72 lines)

main() inspects argv[1] and dispatches to the correct function. For set and in it converts the time argument into an absolute target time, joins the remaining words into the message, starts the alarm and prints the confirmation line. For list, clear and cancel it calls the matching function. All error messages for bad input are produced here.

Module 2: Alarm module (alarm_set.c, 133 lines)

Function	Responsibility
parse_time_spec()	Turn HH:MM or HH:MM:SS into the next matching epoch time, rejecting out-of-range fields and trailing junk
parse_relative()	Turn 90, 30s, 5m, 1h30m into seconds, accepting 1 second to 24 hours
compute_delay()	Return target minus now, never negative
alarm_child()	The body of the child process: setsid, handlers, alarm, wait, ring, record, exit
start_alarm()	Reserve an id, fork, and in the parent record the child PID
cancel_alarm()	Find a PENDING alarm, send SIGTERM, mark CANCELLED
list_alarms()	Print the table, marking vanished processes STALE
clear_finished()	Remove every record that is not PENDING

Module 3: Signal module (signal_handler.c, 56 lines)

Function	Responsibility
on_sigalrm(), on_sigterm()	Handlers that set g_alarm_fired or g_terminate
install_handlers()	Register the handlers with sigaction() for SIGALRM, SIGTERM and SIGINT

Function	Responsibility
wait_for_signal()	Block the signals, loop in sigsuspend() until a flag is set, restore the mask
ring()	Print three bell characters and the framed banner with the time and message

Module 4: Persistence and logging (logger.c, 142 lines)

Function	Responsibility
lock_db() and unlock_db()	Acquire and release the exclusive flock on the lock file
load_nolock() and save_nolock()	Read and atomically rewrite the database while the lock is held
db_add()	Assign the next id and append a record
db_set_status(), db_set_pid()	Update one field of one record
db_remove_finished()	Keep only PENDING records
db_load()	Locked read for list and cancel
log_event()	Append a timestamped, PID-tagged line to the log

Shared header (alarm.h, 45 lines)

The header defines the constants MAX_MSG (128) and MAX_ALARM (64), the file names, the alarm_rec_t record type and the prototypes of every function that crosses a module boundary.

5.5 Input and Output Design

Command syntax

Command	Meaning	Example
alarm_clock set HH:MM[:SS] [message]	Alarm at the next occurrence of a clock time	alarm_clock set 07:30 Wake up

Command	Meaning	Example
alarm_clock in DURATION [message]	Alarm after a delay: 90, 30s, 5m, 2h, 1h30m10s	alarm_clock in 25m Pomodoro done
alarm_clock list	Show all alarms	alarm_clock list
alarm_clock cancel ID	Cancel a pending alarm	alarm_clock cancel 2
alarm_clock clear	Remove finished alarms	alarm_clock clear
alarm_clock help	Print the usage text	alarm_clock help

Output messages

Situation	Output	Stream
Alarm created	Alarm #1 set for 2026-10-13 03:51:28 (in 3 s)	stdout
Alarm rings	Three bells and a framed banner with time and message	stdout
Alarm cancelled	Alarm #3 cancelled.	stdout
Alarms cleared	Removed 3 finished alarm(s).	stdout
Empty list	No alarms.	stdout
Bad clock time	error: invalid time ‘25:61’ (use HH:MM or HH:MM:SS)	stderr
Bad duration	error: invalid duration ‘0s’ (1s..24h)	stderr
Missing argument	error: time argument missing	stderr
Unknown command	error: unknown command ‘foo’ followed by the usage text	stderr
Cancel twice	alarm #3 is already CANCELLED	stderr
Cancel unknown id	no such alarm: #99	stderr

Error messages go to the standard error stream and the program exits with status 1, so shell scripts can detect failure with the usual exit-status test.

File formats

The database ~/.alarm_clock.db has one line per alarm with five fields separated by a vertical bar: id, PID of the alarm process, target time as seconds since the epoch, status, and the message. A vertical bar or newline typed in a message is replaced by a slash when the record is saved, so the format stays unambiguous.

```
2|141|1791863545|PENDING|Stand up and stretch
```

The log ~/.alarm_clock.log has one line per event, with a timestamp, the PID of the process that wrote it and a description.

```
[2026-10-13 03:51:25] [pid 139] alarm #1 armed, delay=3s
[2026-10-13 03:51:29] [pid 139] alarm #1 fired: Tea is ready
```

CHAPTER 6 IMPLEMENTATION

6.1 Module-wise Implementation

The implementation follows the module structure of Chapter 5. This section explains how each module was built and in which order, so that a reader can rebuild the project step by step. The complete source is listed in section 6.3.

Step 1: shared definitions (alarm.h)

The header was written first because it fixes the vocabulary of the whole program. It defines the limits MAX_MSG (128 characters per message) and MAX_ALARM (64 alarms), the two hidden file names, the alarm_rec_t record and the prototypes of all cross-module functions. Because every .c file includes it, the compiler checks every call between modules against these prototypes.

Step 2: persistence and logging (logger.c)

Persistence was built second because everything else depends on being able to store and retrieve an alarm. The module is organised in two layers. The lower layer consists of three small static functions that must be called with the lock held: load_nolock() parses the text file into an array of records, save_nolock() writes the array to a temporary file and renames it over the database, and lock_db() takes the exclusive lock. The upper layer

consists of the public functions db_add(), db_set_status(), db_set_pid(), db_remove_finished() and db_load(), each of which follows the same pattern: lock, load, modify, save, unlock. Keeping the pattern identical makes the locking easy to audit.

Step 3: time handling and alarm creation (alarm_set.c)

The pure functions came first: parse_time_spec(), parse_relative() and compute_delay(). They contain no system calls that change state, so they could be unit-tested at once. Then alarm_child() and start_alarm() were added, followed by cancel_alarm(), list_alarms() and clear_finished().

The order of operations inside start_alarm() is important. The record is stored before fork() so that the alarm id exists when the child begins, stdout is flushed before fork() so that buffered text is not printed twice, and the child PID is saved only after fork() returns in the parent, since only then is it known.

Step 4: signals and notification (signal_handler.c)

The signal module is the smallest and most delicate. Two flag variables, two handlers of one line each, install_handlers() to register them with sigaction(), wait_for_signal() containing the mask-and-suspend loop, and ring() to print the notification.

Step 5: command-line front end (main.c)

The front end was written last. It parses the arguments, calls the functions above, and turns failures into messages and exit codes. Because all logic lives in the lower modules, main.c is only 72 lines.

Step 6: build and tests

The Makefile, the unit-test program tests/test_parse.c and the timing script tests/accuracy.sh complete the project. The unit test links the same object code as the real program, so it tests the shipped functions and not a copy.

6.2 Algorithm / Pseudocode

parse_time_spec(text, now) returns target

```
read hour, minute and optional second from text as H:M[:S]
if the format is wrong or any field is out of range: return error
break now into calendar fields (localtime_r)
replace hour, minute, second with the requested values
target = mktime(fields)              (let the library handle DST)
if target <= now: add one day and call mktime again
return target
```

parse_relative(text) returns seconds

```
total = 0; current = 0
for each character c in text:
    if c is a digit:        current = current * 10 + digit
    else if c is h, m or s and a number is pending:
                            total += current * (3600, 60 or 1);  current = 0
    else:                   return error
if a number is still pending: total += current      (bare number = seconds)
if total < 1 or total > 86400: return error
return total
```

start_alarm(target, message)

```
record = (target, status PENDING, message with | and newline replaced)
id = db_add(record)                  (locked; assigns next id)
flush stdout
pid = fork()
if pid < 0:      mark CANCELLED; return error
if pid == 0:     alarm_child(id, target, message)      (never returns)
db_set_pid(id, pid)                  (parent only)
return id
```

alarm_child(id, target, message)

```
setsid()                                (leave the terminal session)
install_handlers()                      (SIGALRM, SIGTERM, SIGINT set flags)
delay = max(0, target - now)
log armed
alarm(delay)                            (kernel will send SIGALRM; a delay of 0
sets the fired flag directly)
wait_for_signal()
if g_alarm_fired:  ring(message); db_set_status(id, DONE); log fired
else:              log terminated
_exit(0)
```

wait_for_signal()

```
set = {SIGALRM, SIGTERM, SIGINT}
sigprocmask(BLOCK, set, old)          (signals now stay pending)
while neither flag is set:
    sigsuspend(old)                   (atomically: unblock, sleep, handler
runs, re-block)
sigprocmask(SETMASK, old)             (restore)
```

Database update cycle (every db_* writer)

```
fd = open(lock file); flock(fd, EXCLUSIVE)        (wait if another process
holds it)
records = parse database file
change the record(s)
write records to database.tmp; rename(database.tmp, database)
flock(fd, UNLOCK); close(fd)
```

cancel_alarm(id)

```
records = db_load()
find record with this id; if none: error no such alarm
if status != PENDING: error already <status>
kill(pid, SIGTERM)                    (ignore ESRCH: process already gone)
db_set_status(id, CANCELLED); log
```

Complexity

Time is dominated by waiting and is not a computational cost. Every database operation reads and rewrites at most 64 lines, so it is O(n) with n at most 64. Memory use is constant: one array of 64 records of about 150 bytes each on the stack during a database operation.

6.3 Core C Implementation

This section lists the complete source code. The four modules and the header total 448 lines.

include/alarm.h

```
#ifndef ALARM_H
#define ALARM_H

#include <signal.h>
#include <sys/types.h>
#include <time.h>

#define MAX_MSG   128
#define MAX_ALARM 64
#define DB_NAME   ".alarm_clock.db"
#define LOG_NAME  ".alarm_clock.log"

typedef struct {
    int    id;
    pid_t  pid;
    time_t target;
    char   status[12];          /* PENDING, DONE, CANCELLED */
    char   msg[MAX_MSG];
} alarm_rec_t;

/* alarm_set.c : time parsing, delay computation, process creation */
int  parse_time_spec(const char *s, time_t now, time_t *target);
int  parse_relative(const char *s, long *secs);
long compute_delay(time_t now, time_t target);
int  start_alarm(time_t target, const char *msg);
int  cancel_alarm(int id);
void list_alarms(void);
void clear_finished(void);
```

```
/* signal_handler.c : SIGALRM / SIGTERM handling */
extern volatile sig_atomic_t g_alarm_fired;
extern volatile sig_atomic_t g_terminate;
void install_handlers(void);
void wait_for_signal(void);
void ring(const char *msg);

/* logger.c : persistence (flock protected) and log file */
int  db_load(alarm_rec_t *recs, int max);
int  db_add(alarm_rec_t *rec);
int  db_set_status(int id, const char *status);
int  db_set_pid(int id, pid_t pid);
int  db_remove_finished(void);
void log_event(const char *fmt, ...);

#endif
```

src/main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include "alarm.h"

static void usage(const char *p)
{
    printf("Alarm Clock - Linux System Programming\n\n"
           "Usage:\n"
           "  %s set <HH:MM[:SS]> [message]   alarm at a clock time\n"
           "  %s in  <30s|5m|1h30m> [message] alarm after a duration\n"
           "  %s list                         show all alarms\n"
           "  %s cancel <id>                  cancel a pending alarm\n"
           "  %s clear                        remove finished alarms\n"
           "  %s help                         show this text\n",
           p, p, p, p, p, p);
}

static void join_msg(int argc, char **argv, int from, char *out, size_t n)
{
    out[0] = '\0';
    for (int i = from; i < argc; i++) {
        if (out[0]) strncat(out, " ", n - strlen(out) - 1);
```

```
        strncat(out, argv[i], n - strlen(out) - 1);
    }
}

int main(int argc, char **argv)
{
    char msg[MAX_MSG];
    time_t now = time(NULL), target;

    if (argc < 2 || strcmp(argv[1], "help") == 0) { usage(argv[0]); return
argc < 2; }

    if (strcmp(argv[1], "set") == 0 || strcmp(argv[1], "in") == 0) {
        if (argc < 3) { fprintf(stderr, "error: time argument missing\n");
return 1; }
        if (argv[1][0] == 's') {
            if (parse_time_spec(argv[2], now, &target) < 0) {
                fprintf(stderr, "error: invalid time '%s' (use HH:MM or
HH:MM:SS)\n", argv[2]);
                return 1;
            }
        } else {
            long secs;
            if (parse_relative(argv[2], &secs) < 0) {
                fprintf(stderr, "error: invalid duration '%s' (1s..24h)\n",
argv[2]);
                return 1;
            }
            target = now + secs;
        }
        join_msg(argc, argv, 3, msg, sizeof msg);
        int id = start_alarm(target, msg);
        if (id < 0) return 1;
        char ts[32]; struct tm tm;
        localtime_r(&target, &tm);
        strftime(ts, sizeof ts, "%Y-%m-%d %H:%M:%S", &tm);
        printf("Alarm #%d set for %s (in %ld s)\n", id, ts,
compute_delay(now, target));
        return 0;
    }
    if (strcmp(argv[1], "list") == 0)  { list_alarms(); return 0; }
    if (strcmp(argv[1], "clear") == 0) { clear_finished(); return 0; }
    if (strcmp(argv[1], "cancel") == 0) {
```

```
        if (argc < 3) { fprintf(stderr, "error: alarm id missing\n"); return
1; }
        int id = atoi(argv[2]);
        if (id <= 0 || cancel_alarm(id) < 0) return 1;
        printf("Alarm #%d cancelled.\n", id);
        return 0;
    }
    fprintf(stderr, "error: unknown command '%s'\n", argv[1]);
    usage(argv[0]);
    return 1;
}
```

Commentary on main.c: the program uses a simple if-chain rather than a table of commands because there are only six commands and the chain reads like the usage text. The variable now is read once at the top, so the delay printed in the confirmation and the delay used by the child derive from the same instant. The message is built by join_msg(), which uses the bounded strncat() so that a very long message is truncated and not overflowed.

src/signal_handler.c

```
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include "alarm.h"

volatile sig_atomic_t g_alarm_fired = 0;
volatile sig_atomic_t g_terminate   = 0;

/* Async-signal-safe handlers: they only set flags. */
static void on_sigalrm(int sig) { (void)sig; g_alarm_fired = 1; }
static void on_sigterm(int sig) { (void)sig; g_terminate = 1; }

void install_handlers(void)
{
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;                /* no SA_RESTART: sigsuspend must wake
*/
```

```
    sa.sa_handler = on_sigalrm;
    sigaction(SIGALRM, &sa, NULL);

    sa.sa_handler = on_sigterm;
    sigaction(SIGTERM, &sa, NULL);
    sigaction(SIGINT,  &sa, NULL);
}

/* Sleep until SIGALRM or SIGTERM arrives, without a race window. */
void wait_for_signal(void)
{
    sigset_t block, old;
    sigemptyset(&block);
    sigaddset(&block, SIGALRM);
    sigaddset(&block, SIGTERM);
    sigaddset(&block, SIGINT);
    sigprocmask(SIG_BLOCK, &block, &old);       /* close the race window   */
    while (!g_alarm_fired && !g_terminate)
        sigsuspend(&old);                       /* atomically unblock+wait */
    sigprocmask(SIG_SETMASK, &old, NULL);
}

void ring(const char *msg)
{
    char ts[32];
    time_t now = time(NULL);
    struct tm tm;
    localtime_r(&now, &tm);
    strftime(ts, sizeof ts, "%H:%M:%S", &tm);
    for (int i = 0; i < 3; i++) { fputc('\a', stdout); fflush(stdout);
usleep(200000); }
    printf("\n+----------------------------------------+\n");
    printf("|  ALARM!  Time is %-26s|\n", ts);
    printf("|  %-42.42s|\n", msg[0] ? msg : "Wake up!");
    printf("+----------------------------------------+\n");
    fflush(stdout);
}
```

Commentary on signal_handler.c: the memset() and sigemptyset() calls give struct sigaction a defined starting state. The flags are zero on purpose. With SA_RESTART set, an interrupted system call would be restarted automatically; sigsuspend() is never restarted, but keeping the flag off makes the intent explicit and protects against later changes. Note

that the handler for SIGTERM is also installed for SIGINT, so pressing Ctrl+C in a foreground test ends the wait politely instead of killing the process mid-write. The banner width is fixed so that the frame lines up, and a message longer than 42 characters is truncated by the precision in the format %-42.42s.

src/alarm_set.c

```
#include <ctype.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <unistd.h>
#include "alarm.h"

/* "HH:MM" or "HH:MM:SS" -> next occurrence as an epoch time. */
int parse_time_spec(const char *s, time_t now, time_t *target)
{
    int h, m, sec = 0, n;
    char junk;
    n = sscanf(s, "%d:%d:%d%c", &h, &m, &sec, &junk);
    if (n < 2 || n > 3) return -1;
    if (h < 0 || h > 23 || m < 0 || m > 59 || sec < 0 || sec > 59) return -1;

    struct tm tm;
    localtime_r(&now, &tm);
    tm.tm_hour = h; tm.tm_min = m; tm.tm_sec = sec; tm.tm_isdst = -1;
    time_t t = mktime(&tm);
    if (t <= now) { tm.tm_mday += 1; tm.tm_isdst = -1; t = mktime(&tm); }
    *target = t;
    return 0;
}

/* "90", "30s", "5m", "2h", "1h30m10s" -> seconds. */
int parse_relative(const char *s, long *secs)
{
    long total = 0, cur = 0; int have = 0, any = 0;
    for (const char *p = s; *p; p++) {
        if (isdigit((unsigned char)*p)) { cur = cur * 10 + (*p - '0'); have =
1; }
        else if (have && (*p == 'h' || *p == 'm' || *p == 's')) {
```

```
            total += cur * (*p == 'h' ? 3600 : *p == 'm' ? 60 : 1);
            cur = 0; have = 0; any = 1;
        } else return -1;
    }
    if (have) { total += cur; any = 1; }      /* bare number = seconds */
    if (!any || total <= 0 || total > 86400) return -1;
    *secs = total;
    return 0;
}

long compute_delay(time_t now, time_t target)
{
    long d = (long)difftime(target, now);
    return d < 0 ? 0 : d;
}

/* Child body: arm the kernel timer, sleep, notify, record, exit. */
static void alarm_child(int id, time_t target, const char *msg)
{
    setsid();                                 /* detach from terminal session
*/
    install_handlers();
    long delay = compute_delay(time(NULL), target);
    log_event("alarm #%d armed, delay=%lds", id, delay);
    if (delay > 0) alarm((unsigned)delay); else g_alarm_fired = 1;
/* kernel timer -> SIGALRM */
    wait_for_signal();
    if (g_alarm_fired) {
        ring(msg);
        db_set_status(id, "DONE");
        log_event("alarm #%d fired: %s", id, msg);
    } else {
        log_event("alarm #%d terminated by signal before expiry", id);
    }
    _exit(0);
}

int start_alarm(time_t target, const char *msg)
{
    alarm_rec_t r;
    memset(&r, 0, sizeof r);
    r.target = target;
    strcpy(r.status, "PENDING");
```

```
    strncpy(r.msg, msg, MAX_MSG - 1);
    for (char *p = r.msg; *p; p++) if (*p == '|' || *p == '\n') *p = '/';

    int id = db_add(&r);                    /* reserve an id first */
    if (id < 0) { fprintf(stderr, "error: cannot store alarm\n"); return -1;
}

    fflush(stdout);
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); db_set_status(id, "CANCELLED"); return -1;
}
    if (pid == 0) alarm_child(id, target, r.msg);

    db_set_pid(id, pid);                    /* parent records child pid */
    return id;
}

int cancel_alarm(int id)
{
    alarm_rec_t all[MAX_ALARM];
    int n = db_load(all, MAX_ALARM);
    for (int i = 0; i < n; i++) {
        if (all[i].id != id) continue;
        if (strcmp(all[i].status, "PENDING") != 0) {
            fprintf(stderr, "alarm #%d is already %s\n", id, all[i].status);
            return -1;
        }
        if (kill(all[i].pid, SIGTERM) < 0 && errno != ESRCH)
            perror("kill");
        db_set_status(id, "CANCELLED");
        log_event("alarm #%d cancelled by user", id);
        return 0;
    }
    fprintf(stderr, "no such alarm: #%d\n", id);
    return -1;
}

void list_alarms(void)
{
    alarm_rec_t all[MAX_ALARM];
    int n = db_load(all, MAX_ALARM);
    if (n == 0) { puts("No alarms."); return; }
    printf("%-4s %-8s %-19s %-10s %s\n", "ID", "PID", "TIME", "STATUS",
```

```
"MESSAGE");
    for (int i = 0; i < n; i++) {
        char ts[32]; struct tm tm;
        if (strcmp(all[i].status, "PENDING") == 0 &&
            kill(all[i].pid, 0) < 0 && errno == ESRCH)
            strcpy(all[i].status, "STALE");   /* process vanished */
        localtime_r(&all[i].target, &tm);
        strftime(ts, sizeof ts, "%Y-%m-%d %H:%M:%S", &tm);
        printf("%-4d %-8d %-19s %-10s %s\n", all[i].id, (int)all[i].pid,
               ts, all[i].status, all[i].msg);
    }
}

void clear_finished(void)
{
    int k = db_remove_finished();
    printf("Removed %d finished alarm(s).\n", k < 0 ? 0 : k);
}
```

Commentary on alarm_set.c: three details deserve attention. First, tm_isdst is set to -1 before each mktime() call, which tells the library to work out for itself whether daylight-saving time applies on the target date. Second, the sscanf() format in parse_time_spec() ends with a character conversion; if any character follows the seconds field, the call returns 4 and the input is rejected, which is how 12:30:15x is detected. Third, list_alarms() sends signal 0 with kill(); signal 0 delivers nothing but reports whether the process exists, which is the standard existence test.

src/logger.c

```
#include <errno.h>
#include <fcntl.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/file.h>
#include <unistd.h>
#include "alarm.h"

static void path_of(const char *name, char *out, size_t n)
{
    const char *home = getenv("HOME");
```

```
    snprintf(out, n, "%s/%s", home ? home : ".", name);
}

/* Take an exclusive advisory lock on a separate lock file. */
static int lock_db(void)
{
    char p[256];
    path_of(".alarm_clock.lock", p, sizeof p);
    int fd = open(p, O_CREAT | O_RDWR, 0600);
    if (fd < 0) return -1;
    if (flock(fd, LOCK_EX) < 0) { close(fd); return -1; }
    return fd;
}

static void unlock_db(int fd) { flock(fd, LOCK_UN); close(fd); }

static int load_nolock(alarm_rec_t *r, int max)
{
    char p[256], line[256];
    path_of(DB_NAME, p, sizeof p);
    FILE *f = fopen(p, "r");
    if (!f) return 0;
    int n = 0;
    while (n < max && fgets(line, sizeof line, f)) {
        alarm_rec_t a;
        long long t;
        memset(&a, 0, sizeof a);
        if (sscanf(line, "%d|%d|%lld|%11[^|]", &a.id, &a.pid, &t, a.status)
!= 4)
            continue;
        a.target = (time_t)t;
        char *q = line;                      /* message follows the 4th '|'
*/
        for (int k = 0; k < 4 && q; k++) { q = strchr(q, '|'); if (q) q++; }
        strncpy(a.msg, q ? q : "", MAX_MSG - 1);
        a.msg[strcspn(a.msg, "\n")] = '\0';
        r[n++] = a;
    }
    fclose(f);
    return n;
}

static int save_nolock(const alarm_rec_t *r, int n)
```

```
{
    char p[256], tmp[260];
    path_of(DB_NAME, p, sizeof p);
    snprintf(tmp, sizeof tmp, "%s.tmp", p);
    FILE *f = fopen(tmp, "w");
    if (!f) return -1;
    for (int i = 0; i < n; i++)
        fprintf(f, "%d|%d|%lld|%s|%s\n", r[i].id, (int)r[i].pid,
                (long long)r[i].target, r[i].status, r[i].msg);
    fclose(f);
    return rename(tmp, p);                    /* atomic replace */
}

int db_load(alarm_rec_t *recs, int max)
{
    int fd = lock_db();
    if (fd < 0) return 0;
    int n = load_nolock(recs, max);
    unlock_db(fd);
    return n;
}

int db_add(alarm_rec_t *rec)
{
    alarm_rec_t all[MAX_ALARM];
    int fd = lock_db();
    if (fd < 0) return -1;
    int n = load_nolock(all, MAX_ALARM), maxid = 0;
    for (int i = 0; i < n; i++) if (all[i].id > maxid) maxid = all[i].id;
    if (n >= MAX_ALARM) { unlock_db(fd); return -1; }
    rec->id = maxid + 1;
    all[n++] = *rec;
    int rc = save_nolock(all, n);
    unlock_db(fd);
    return rc == 0 ? rec->id : -1;
}

static int update_field(int id, const char *status, pid_t pid)
{
    alarm_rec_t all[MAX_ALARM];
    int fd = lock_db();
    if (fd < 0) return -1;
    int n = load_nolock(all, MAX_ALARM), found = 0;
```

```
    for (int i = 0; i < n; i++)
        if (all[i].id == id) {
            if (status) { strncpy(all[i].status, status, sizeof all[i].status
- 1); }
            if (pid > 0) all[i].pid = pid;
            found = 1;
        }
    if (found) save_nolock(all, n);
    unlock_db(fd);
    return found ? 0 : -1;
}

int db_set_status(int id, const char *status) { return update_field(id,
status, 0); }
int db_set_pid(int id, pid_t pid)             { return update_field(id, NULL,
pid); }

int db_remove_finished(void)
{
    alarm_rec_t all[MAX_ALARM];
    int fd = lock_db();
    if (fd < 0) return -1;
    int n = load_nolock(all, MAX_ALARM), k = 0;
    for (int i = 0; i < n; i++)
        if (strcmp(all[i].status, "PENDING") == 0) all[k++] = all[i];
    save_nolock(all, k);
    unlock_db(fd);
    return n - k;
}

void log_event(const char *fmt, ...)
{
    char p[256], ts[32];
    path_of(LOG_NAME, p, sizeof p);
    FILE *f = fopen(p, "a");
    if (!f) return;
    time_t now = time(NULL);
    struct tm tm;
    localtime_r(&now, &tm);
    strftime(ts, sizeof ts, "%Y-%m-%d %H:%M:%S", &tm);
    fprintf(f, "[%s] [pid %d] ", ts, (int)getpid());
    va_list ap;
    va_start(ap, fmt);
```

```
    vfprintf(f, fmt, ap);
    va_end(ap);
    fputc('\n', f);
    fclose(f);
}
```

Commentary on logger.c: every public database function opens its own file descriptor for the lock file and closes it before returning. Closing the descriptor releases the lock even if the function returned early, so a lock can never be left held by a finished function. The scan-set conversion in the sscanf() call reads at most 11 characters of the status, which fits the 12-byte status field with its terminator and so protects the buffer. The log file is opened in append mode; on Linux, each small write in append mode goes to the end of the file, so lines from different alarm processes do not overwrite one another.

Makefile

```
CC      = gcc
CFLAGS  = -Wall -Wextra -std=c11 -D_GNU_SOURCE -g -O0 -Iinclude
SRC     = src/main.c src/alarm_set.c src/signal_handler.c src/logger.c
OBJ     = $(SRC:.c=.o)
TARGET  = alarm_clock

all: $(TARGET)

$(TARGET): $(OBJ)
        $(CC) $(CFLAGS) -o $@ $(OBJ)

%.o: %.c include/alarm.h
        $(CC) $(CFLAGS) -c $< -o $@

clean:
        rm -f $(OBJ) $(TARGET)

.PHONY: all clean
```

tests/test_parse.c (unit tests)

```
#include <stdio.h>
#include <string.h>
#include <time.h>
#include "alarm.h"
```

```
static int pass, fail;
#define CHECK(name, cond) do { if (cond) { pass++; printf("PASS  %s\n",
name); } \
                               else { fail++; printf("FAIL  %s\n", name); } }
while (0)

int main(void)
{
    long s; time_t t, now = time(NULL);
    struct tm tm;

    CHECK("relative 30s",       parse_relative("30s", &s) == 0 && s == 30);
    CHECK("relative 5m",        parse_relative("5m", &s) == 0 && s == 300);
    CHECK("relative 1h30m",     parse_relative("1h30m", &s) == 0 && s ==
5400);
    CHECK("relative 1h2m3s",    parse_relative("1h2m3s", &s) == 0 && s ==
3723);
    CHECK("relative bare 90",   parse_relative("90", &s) == 0 && s == 90);
    CHECK("relative 0s rejected",   parse_relative("0s", &s) < 0);
    CHECK("relative 25h rejected",  parse_relative("25h", &s) < 0);
    CHECK("relative junk rejected", parse_relative("5x", &s) < 0);
    CHECK("relative empty rejected",parse_relative("", &s) < 0);

    CHECK("HH:MM valid",        parse_time_spec("07:30", now, &t) == 0);
    localtime_r(&t, &tm);
    CHECK("HH:MM fields",       tm.tm_hour == 7 && tm.tm_min == 30 &&
tm.tm_sec == 0);
    CHECK("HH:MM in future",    t > now);
    CHECK("HH:MM:SS valid",     parse_time_spec("23:59:59", now, &t) == 0);
    CHECK("hour 24 rejected",   parse_time_spec("24:00", now, &t) < 0);
    CHECK("minute 60 rejected", parse_time_spec("10:60", now, &t) < 0);
    CHECK("letters rejected",   parse_time_spec("ab:cd", now, &t) < 0);
    CHECK("single field rejected", parse_time_spec("12", now, &t) < 0);
    CHECK("trailing junk rejected", parse_time_spec("12:30:15x", now, &t) <
0);

    /* a time earlier today rolls over to tomorrow */
    localtime_r(&now, &tm);
    char past[16];
    snprintf(past, sizeof past, "%02d:%02d:%02d", tm.tm_hour, tm.tm_min,
tm.tm_sec);
    CHECK("past time rolls to tomorrow",
          parse_time_spec(past, now, &t) == 0 && t > now && t - now > 86300);
```

```
    CHECK("delay positive",     compute_delay(100, 160) == 60);
    CHECK("delay clamps at 0",  compute_delay(200, 100) == 0);

    printf("\n%d passed, %d failed\n", pass, fail);
    return fail != 0;
}
```

tests/accuracy.sh

```
#!/bin/sh
# Arm alarms for 2,5,8 and 12 seconds and compare firing time with target.
for d in 2 5 8 12; do ./alarm_clock in ${d}s "T+$d" >/dev/null; done
sleep 14
grep -E "armed|fired" "$HOME/.alarm_clock.log"
```

6.4 Linux System Calls / APIs Used

The table groups the interfaces by the kernel service they use and states the rule that the code follows for each. Section 4.3 gives the prototypes; this section records how each call is used and how failures are handled.

Service	Calls	How the code uses them	Failure handling
Process creation	fork()	One call per alarm, after the record is stored and stdout is flushed	Negative result: perror, mark the alarm CANCELLED, return an error
Session control	setsid()	First action of the child	Result ignored: the child of a fork is never a process-group leader, so the call is expected to succeed
Timer	alarm()	Single call in the child with the computed delay in seconds	Cannot fail; a delay of 0 is handled separately, see the note below

Service	Calls	How the code uses them	Failure handling
Signal registration	sigaction()	SIGALRM, SIGTERM and SIGINT with plain handlers and no flags	Valid signals only, so failure is not expected
Signal blocking and waiting	sigprocmask(), sigsuspend()	Block, test flags, suspend, restore	sigsuspend always returns -1 with EINTR, which is the normal wake-up and is not an error
Signal sending	kill()	SIGTERM for cancel; signal 0 for the existence test	ESRCH (no such process) is tolerated; other errors are reported with perror
Time	time(), localtime_r(), mktime(), strftime(), difftime()	Parsing, delay and display	Invalid input rejected before any call
Locking	open(), flock(), close()	Exclusive lock around each database cycle	If the lock cannot be taken, the operation reports failure
Atomic replacement	fopen(), fclose(), rename()	Write the temporary file, then rename it over the database	A failed fopen or rename returns -1
Termination	_exit()	End of the child	Does not return

Note on a zero delay: alarm(0) does not start a timer; it cancels any pending one. A target that is already due can only occur if the process was delayed between the parent computing the target and the child computing the delay. That case is possible, for example when an alarm for the next second is started at the very end of a second, so alarm_child() sets the fired flag directly when the delay is zero. The wait loop then returns at once instead of sleeping forever.

Choice of sigaction() over signal()

The proposal lists signal() and sigaction() together. The implementation uses sigaction(), because the behaviour of signal() varies between Unix systems: some reset the handler to the default after one delivery, and the interaction with interrupted system calls differs. sigaction() has one precisely defined behaviour, and it lets the program choose the flags and the mask explicitly.

6.5 Process / Thread Implementation

Process creation and ownership

Each alarm is a process created by fork(). The sequence of ownership is as follows. The shell forks a process that executes alarm_clock. That process calls fork() again to create the alarm child and exits almost at once. At this moment the child's parent has died, so the kernel re-parents the child to the init process (PID 1, or the nearest subreaper such as a user systemd instance). The child is thereafter reaped by that process when it exits, so no zombie remains. Because the alarm_clock parent does not call wait(), it never blocks, and the user's prompt returns immediately.

Process states

A waiting alarm is in the interruptible sleep state, shown as S by ps. The kernel records the function in which it sleeps, and ps shows it in the WCHAN column. Measurements taken from a live alarm show state S in sigsuspend, one thread, no CPU consumption and a signal-catch mask of 0x6002. The mask is the sum of three bits: 0x2 for SIGINT (signal 2), 0x2000 for SIGALRM (signal 14) and 0x4000 for SIGTERM (signal 15), which proves that exactly the three intended handlers are installed.

Phase	Process state	Visible evidence
Before fork	Running (R)	The parent is executing main()
Waiting for the alarm	Interruptible sleep (S)	ps shows STAT Ss and WCHAN sigsuspend
Signal delivered	Running briefly	Handler sets the flag; process returns to the loop
Ringing	Running and sleeping in 200 ms steps	Three bell characters, then the banner

Phase	Process state	Visible evidence
Finished	Exited	The process no longer appears in ps; the status is DONE

The letter s in the STAT value Ss means the process is a session leader, which is direct evidence that setsid() worked.

Handling lost or killed processes

A user could end an alarm process with kill -9, or the machine could be restarted. The database would then still say PENDING for a process that no longer exists. list_alarms() detects this with kill(pid, 0) and shows the status STALE, so the table never misleads the user. A known limitation is that the kernel eventually reuses process identifiers, so a very old stale record could, in principle, name an unrelated process. The cancel command does not check the process start time, which is acceptable for the short-lived alarms the program is designed for and is listed as future work.

Why there are no threads

The reasons were given in section 4.4. In short, one timer per process and process-directed signals make one process per alarm the simplest correct design, and the program is therefore single-threaded in every process.

6.6 IPC / Synchronization Implementation

The processes of the alarm clock communicate by four channels, each chosen for a specific purpose.

Channel	Between	Purpose
Signals (SIGTERM)	cancel command to alarm child	Request early termination
Signals (SIGALRM)	Kernel to alarm child	Report that the timer expired
Shared files (database, log)	All processes	Persistent state and history
Process identifier	Parent to database to cancel command	Name the target of kill()

Race conditions considered

Race	What could go wrong	How it is prevented
Flag test versus sleep	SIGALRM arrives between the test and the sleep; the process sleeps forever	Signals are blocked first; sigsuspend() unblocks and sleeps atomically
Alarm finishes before the PID is saved	Child marks DONE before the parent records the PID	The id is reserved first; status and PID are separate fields updated by id, under the lock
Two commands update the database together	One update overwrites the other	flock(LOCK_EX) serialises the whole load-modify-save cycle
Reader sees a half-written file	list shows garbage or loses alarms	The writer fills a temporary file and rename() replaces the database atomically
Two processes choose the same id	Duplicate ids	The next id is computed inside the locked section of db_add()
Duplicate output after fork	Buffered text printed by both processes	fflush(stdout) before fork(); child ends with _exit()
Cancel arrives before the child installs handlers	SIGTERM default action kills the child with no log entry	Acceptable: the result is still a terminated alarm; the status is set CANCELLED by the cancel command itself

Why advisory locking is sufficient

flock() is advisory: it only works if every participant asks for the lock. In this program every access to the database passes through functions in logger.c that take the lock, so the rule is enforced by code structure. The lock is released automatically if the process holding it is killed, so a crashed alarm cannot block the others.

6.7 Memory Management Implementation

The program deliberately uses no dynamic memory. There is no malloc() or free() anywhere in the source, so there is no possibility of a memory leak or a use-after-free error.

Memory area	Contents	Size
Code (text segment)	The compiled program, shared with other alarm processes	A few tens of kilobytes at -O0
Global data	Two sig_atomic_t flags	8 bytes
Stack, main	msg buffer, timestamps	About 200 bytes
Stack, database functions	alarm_rec_t all[64] plus line buffers	64 records of 160 bytes, about 10 kB
Kernel structures	Process table entry, signal tables, timer	Managed by the kernel

The record structure holds 156 bytes of fields, which the compiler pads to 160 bytes on x86-64 so that the 8-byte time field stays aligned. A full table of 64 records therefore occupies about 10 kB on the stack, far below the default 8 MB stack limit.

Copy-on-write after fork

After fork(), the parent and child share all physical pages until one of them writes. The code pages are read-only and stay shared for the life of the alarms, which is why many alarm processes together use much less memory than the sum of their sizes. The measured resident set of one waiting alarm is 1064 kB, of which a large part is shared library code.

Buffer safety

Every write into a fixed buffer is bounded: strncpy and strncat with explicit limits, snprintf with the buffer size, a width-limited scan set in sscanf, and the precision in the banner format. Messages longer than the limit are truncated, never overflowed. The program is therefore safe against over-long command-line arguments and corrupted database lines.

6.8 Optional User Interface Implementation

The project provides a text user interface only; a graphical interface is outside the scope. The interface consists of four parts.

- **Usage screen.** alarm_clock help, or alarm_clock with no arguments, prints the six commands with their arguments aligned in columns.

- **Confirmation line.** After set or in, the program prints the alarm number, the absolute date and time, and the delay in seconds, so the user can confirm that the time was understood correctly.
- **Alarm table.** list prints a header and one aligned row per alarm with ID, PID, time, status and message.
- **Notification.** When an alarm rings, the program sends three bell characters (ASCII 7) 200 ms apart and prints a framed banner showing the current time and the message. The frame is 46 characters wide and the message is cut to 42 characters.

Possible extensions of the interface

The notification code is isolated in the single function ring(), so it can be replaced without touching the timer logic. Two natural extensions are a desktop pop-up through the notify-send command and a sound played through an audio utility. Both would be launched from the child with fork() and exec(), and both are mentioned in the proposal as optional; neither is part of the tested build. A GTK or Qt front end could call the same alarm_set.c functions in the same way.

CHAPTER 7 EXECUTION AND DEMONSTRATION

All results in this chapter were produced by running the program described in Chapter 6 on Ubuntu 24.04 (kernel 6.x, x86-64) with GCC 13.3. The terminal transcripts are real output; only the bell characters are written as BEL because they cannot be printed.

7.1 Compilation in Ubuntu

Project layout

```
alarm_clock/
|-- Makefile
|-- include/
|   `-- alarm.h
|-- src/
|   |-- main.c
|   |-- alarm_set.c
|   |-- signal_handler.c
|   `-- logger.c
`-- tests/
    |-- test_parse.c
    `-- accuracy.sh
```

Preparing the environment

```
$ sudo apt update
$ sudo apt install build-essential
$ gcc --version | head -1
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
$ uname -sr
Linux 6.x
```

Building

```
$ make
gcc -Wall -Wextra -std=c11 -D_GNU_SOURCE -g -O0 -Iinclude -c src/main.c -o
src/main.o
gcc -Wall -Wextra -std=c11 -D_GNU_SOURCE -g -O0 -Iinclude -c src/alarm_set.c
-o src/alarm_set.o
gcc -Wall -Wextra -std=c11 -D_GNU_SOURCE -g -O0 -Iinclude -c
src/signal_handler.c -o src/signal_handler.o
gcc -Wall -Wextra -std=c11 -D_GNU_SOURCE -g -O0 -Iinclude -c src/logger.c -o
src/logger.o
gcc -Wall -Wextra -std=c11 -D_GNU_SOURCE -g -O0 -Iinclude -o alarm_clock
src/main.o src/alarm_set.o src/signal_handler.o src/logger.o
```

The build ends without a single warning under -Wall -Wextra. Running make again prints that there is nothing to be done, because the object files are newer than the sources; touching alarm.h rebuilds all four modules, as the pattern rule intends. To start again from scratch, run make clean followed by make.

Building and running the unit tests

```
$ gcc -Wall -Wextra -std=c11 -D_GNU_SOURCE -Iinclude -o tests/test_parse \
      tests/test_parse.c src/alarm_set.c src/signal_handler.c src/logger.c
$ ./tests/test_parse
```

7.2 Execution Commands

Purpose	Command	Result
Show help	./alarm_clock help	Usage text
Alarm after a delay	./alarm_clock in 3s "Tea is ready"	Alarm #n confirmation; prompt returns at once
Alarm at a clock time	./alarm_clock set 07:30 "Wake up"	Next 07:30 is scheduled, today or tomorrow
Alarm with seconds	./alarm_clock set 18:45:30 "Meeting"	Exact-second alarm
Composite duration	./alarm_clock in 1h30m "Break"	Alarm in 5400 seconds
List alarms	./alarm_clock list	Table of all alarms
Cancel	./alarm_clock cancel 3	Pending alarm 3 is ended
Remove finished	./alarm_clock clear	DONE, CANCELLED and STALE records removed
Watch the log	tail -f ~/.alarm_clock.log	Live event stream
Observe a waiting alarm	ps -o pid,ppid,sid,tty,stat,pcpu,rss,wchan:14,cmd -C alarm_clock	State, memory and kernel wait point

7.3 Sample Inputs and Outputs

Test 1: help screen

```
$ ./alarm_clock help
Alarm Clock - Linux System Programming

Usage:
  ./alarm_clock set <HH:MM[:SS]> [message]   alarm at a clock time
  ./alarm_clock in  <30s|5m|1h30m> [message] alarm after a duration
  ./alarm_clock list                         show all alarms
  ./alarm_clock cancel <id>                  cancel a pending alarm
  ./alarm_clock clear                        remove finished alarms
  ./alarm_clock help                         show this text
```

Test 2: three alarms, list, cancel, ring

Three alarms are created in quick succession. The prompt returns after each command, showing that the work happens in background processes.

```
$ ./alarm_clock in 3s "Tea is ready"
Alarm #1 set for 2026-10-13 03:51:28 (in 3 s)
$ ./alarm_clock in 1m "Stand up and stretch"
Alarm #2 set for 2026-10-13 03:52:25 (in 60 s)
$ ./alarm_clock in 10m "Cancel me"
Alarm #3 set for 2026-10-13 04:01:25 (in 600 s)
$ ./alarm_clock list
ID   PID      TIME                STATUS     MESSAGE
1    139      2026-10-13 03:51:28 PENDING    Tea is ready
2    141      2026-10-13 03:52:25 PENDING    Stand up and stretch
3    143      2026-10-13 04:01:25 PENDING    Cancel me
$ ./alarm_clock cancel 3
Alarm #3 cancelled.
$ ./alarm_clock cancel 3
alarm #3 is already CANCELLED
$ ./alarm_clock cancel 99
no such alarm: #99
```

Three seconds after the first command, alarm 1 rings in the same terminal:

```
BEL BEL BEL
+--------------------------------------------+
|  ALARM!  Time is 03:51:28                  |
|  Tea is ready                              |
+--------------------------------------------+
```

The list now shows the three different end states side by side:

```
$ ./alarm_clock list
ID   PID      TIME                STATUS     MESSAGE
1    139      2026-10-13 03:51:28 DONE       Tea is ready
2    141      2026-10-13 03:52:25 PENDING    Stand up and stretch
3    143      2026-10-13 04:01:25 CANCELLED  Cancel me
```

Test 3: invalid input

```
$ ./alarm_clock set 25:61
error: invalid time '25:61' (use HH:MM or HH:MM:SS)
$ ./alarm_clock set abc
error: invalid time 'abc' (use HH:MM or HH:MM:SS)
$ ./alarm_clock in 0s
error: invalid duration '0s' (1s..24h)
$ ./alarm_clock in 99h
error: invalid duration '99h' (1s..24h)
$ ./alarm_clock set
error: time argument missing
$ ./alarm_clock foo
error: unknown command 'foo'
(usage text follows)
```

None of the six invalid commands creates a record or a process, and none crashes the program.

Test 4: alarm by clock time

```
$ date +%H:%M:%S
03:51:30
$ ./alarm_clock set 03:51:34 "Clock-time alarm"
Alarm #4 set for 2026-10-13 03:51:34 (in 4 s)
BEL BEL BEL
+-----------------------------------------+
|  ALARM!  Time is 03:51:34               |
|  Clock-time alarm                       |
+-----------------------------------------+
$ ./alarm_clock clear
Removed 3 finished alarm(s).
$ ./alarm_clock list
ID   PID      TIME                STATUS     MESSAGE
2    141      2026-10-13 03:52:25 PENDING    Stand up and stretch
```

The clear command removed alarm 1 (DONE), alarm 3 (CANCELLED) and alarm 4 (DONE) and kept the one that is still PENDING.

Test 5: log file and database

```
$ cat ~/.alarm_clock.log
[2026-10-13 03:51:25] [pid 139] alarm #1 armed, delay=3s
[2026-10-13 03:51:25] [pid 141] alarm #2 armed, delay=60s
[2026-10-13 03:51:25] [pid 143] alarm #3 armed, delay=600s
[2026-10-13 03:51:25] [pid 143] alarm #3 terminated by signal before expiry
[2026-10-13 03:51:25] [pid 145] alarm #3 cancelled by user
[2026-10-13 03:51:29] [pid 139] alarm #1 fired: Tea is ready
[2026-10-13 03:51:30] [pid 160] alarm #4 armed, delay=4s
[2026-10-13 03:51:34] [pid 160] alarm #4 fired: Clock-time alarm

$ cat ~/.alarm_clock.db
2|141|1791863545|PENDING|Stand up and stretch
```

The log tells the whole story of the cancellation: process 143 received SIGTERM and logged its early termination, and process 145, the cancel command, logged the user request. The fired entry of alarm 1 is stamped one second after its target because the program logs after the three bell characters, which take 0.6 seconds.

Test 6: unit tests

```
$ ./tests/test_parse
PASS  relative 30s
PASS  relative 5m
PASS  relative 1h30m
PASS  relative 1h2m3s
PASS  relative bare 90
PASS  relative 0s rejected
PASS  relative 25h rejected
PASS  relative junk rejected
PASS  relative empty rejected
PASS  HH:MM valid
PASS  HH:MM fields
PASS  HH:MM in future
PASS  HH:MM:SS valid
PASS  hour 24 rejected
PASS  minute 60 rejected
PASS  letters rejected
PASS  single field rejected
PASS  trailing junk rejected
PASS  past time rolls to tomorrow
PASS  delay positive
PASS  delay clamps at 0

21 passed, 0 failed
```

Test 7: timing accuracy with simultaneous alarms

Four alarms are armed together for 2, 5, 8 and 12 seconds (tests/accuracy.sh) and the log is read after 14 seconds.

```
$ ./tests/accuracy.sh
[2026-10-13 04:02:35] [pid 6503] alarm #1 armed, delay=2s
[2026-10-13 04:02:35] [pid 6505] alarm #2 armed, delay=5s
[2026-10-13 04:02:35] [pid 6507] alarm #3 armed, delay=8s
[2026-10-13 04:02:35] [pid 6509] alarm #4 armed, delay=12s
[2026-10-13 04:02:37] [pid 6503] alarm #1 fired: T+2
[2026-10-13 04:02:40] [pid 6505] alarm #2 fired: T+5
[2026-10-13 04:02:43] [pid 6507] alarm #3 fired: T+8
[2026-10-13 04:02:47] [pid 6509] alarm #4 fired: T+12
```

Alarm	Delay requested	Armed at	Fired at	Error
1	2 s	04:02:35	04:02:37	0 s
2	5 s	04:02:35	04:02:40	0 s
3	8 s	04:02:35	04:02:43	0 s
4	12 s	04:02:35	04:02:47	0 s

All four fired in the requested second. The resolution of the measurement is one second, the resolution of alarm() and of the log timestamps.

Test 8: ten alarms created at the same moment

Ten commands were started in parallel from the shell to exercise the file lock.

```
$ for i in 1 2 3 4 5 6 7 8 9 10; do ./alarm_clock in 60s "parallel $i" &
done; wait
$ ./alarm_clock list
ID   PID      TIME                STATUS     MESSAGE
1    6289     2026-10-13 04:01:44 PENDING    parallel 2
2    6291     2026-10-13 04:01:44 PENDING    parallel 3
3    6292     2026-10-13 04:01:44 PENDING    parallel 5
4    6293     2026-10-13 04:01:44 PENDING    parallel 6
5    6294     2026-10-13 04:01:44 PENDING    parallel 8
6    6295     2026-10-13 04:01:44 PENDING    parallel 9
7    6296     2026-10-13 04:01:44 PENDING    parallel 4
8    6297     2026-10-13 04:01:44 PENDING    parallel 1
9    6298     2026-10-13 04:01:44 PENDING    parallel 7
10   6299     2026-10-13 04:01:44 PENDING    parallel 10
```

The result was 10 records, 10 unique ids and 10 unique PIDs. No update was lost, and the order of the messages shows that the commands really did race: the command that was started second obtained id 1. Without the lock, concurrent read-modify-write cycles would have produced duplicate ids or fewer records.

Test 9: capacity limit

Sixty-four alarms were created in a loop. The sixty-fifth command printed error: cannot store alarm and created no process, and the database held exactly 64 records.

Test 10: message handling

A 200-character message was stored as 127 characters. A message containing vertical bars (pipe, and, bars) was saved with slashes in their place (pipe/and/bars), so the database format stayed intact.

Test 11: a vanished process

A record was written by hand with a PID that does not exist, to simulate an alarm process that had died.

```
$ ./alarm_clock list
ID   PID      TIME                 STATUS     MESSAGE
1    999999   2026-10-13 04:10:55 STALE      ghost alarm
$ ./alarm_clock cancel 1
Alarm #1 cancelled.
$ ./alarm_clock clear
Removed 1 finished alarm(s).
$ ./alarm_clock list
No alarms.
```

Summary of test cases

No.	Test	Expected	Result
1	Help screen	Usage text	Pass
2	Duration alarm, list, cancel, ring	Rings once; states DONE, PENDING, CANCELLED	Pass
3	Six invalid inputs	Clear message, exit 1, no process	Pass (6 of 6)
4	Clock-time alarm and clear	Rings at target second; clear removes three	Pass
5	Log and database content	Complete, ordered history	Pass
6	Unit tests	All checks pass	Pass (21 of 21)
7	Four simultaneous alarms	Each fires on time	Pass (4 of 4)
8	Ten parallel creations	Unique ids, no lost record	Pass

No.	Test	Expected	Result
9	Capacity of 64	65th refused cleanly	Pass
10	Long message and bars	Truncated and sanitised	Pass
11	Vanished process	Shown as STALE	Pass

7.4 Screenshots of Execution

The figures below are the terminal captures of the demonstration. Each figure is listed with the command that produces it, so that a reader can reproduce it and capture a screenshot for the submitted copy.

Figure	Description	Command
7.1	Successful build with no warnings	make
7.2	Help screen	./alarm_clock help
7.3	Three alarms created and listed	./alarm_clock in 3s ..., in 1m ..., in 10m ..., list
7.4	Alarm ringing with the banner	wait for the first alarm
7.5	Cancel, repeated cancel and unknown id	./alarm_clock cancel 3, cancel 3, cancel 99
7.6	List showing DONE, PENDING and CANCELLED	./alarm_clock list
7.7	Rejected inputs	set 25:61, in 0s, foo
7.8	Unit test run, 21 of 21	./tests/test_parse
7.9	Process view of waiting alarms	ps -o pid,ppid,sid,tty,stat,pcpu,rss,wchan:14,cmd -C alarm_clock
7.10	Log file	cat ~/.alarm_clock.log

Figure 7.9: process view of waiting alarms

```
$ ps -o pid,ppid,sid,tty,stat,pcpu,rss,wchan:14,cmd -C alarm_clock
  PID  PPID   SID TT       STAT %CPU   RSS WCHAN          CMD
 6289     1  6289 ?        Ss    0.0  1076 sigsuspend.isr ./alarm_clock in
60s parallel 2
 6291     1  6291 ?        Ss    0.0  1080 sigsuspend.isr ./alarm_clock in
60s parallel 3
 6292     1  6292 ?        Ss    0.0  1076 sigsuspend.isr ./alarm_clock in
60s parallel 5
 6293     1  6293 ?        Ss    0.0  1076 sigsuspend.isr ./alarm_clock in
60s parallel 6
```

7.5 Demonstration of OS Concepts

This section connects each observation to the operating-system idea it demonstrates.

Processes: fork() and re-parenting

The PPID column of Figure 7.9 is 1 for every alarm. The processes were created by commands that exited within milliseconds, so their parent died and the kernel re-parented them to the init process. The list command shows ten distinct PIDs for ten alarms, so each alarm is a real, independent process.

Sessions: setsid()

In Figure 7.9 the SID column equals the PID of each process, and the TTY column shows a question mark. A process whose session id equals its own PID is the leader of its own session, and a question mark means it has no controlling terminal. Both facts are direct evidence that setsid() detached the alarm from the shell's session. The s in the STAT value Ss repeats this: it marks a session leader.

Kernel timers and sleeping without CPU

The %CPU column reads 0.0 and the state is S, interruptible sleep. The WCHAN column shows that the process is blocked inside the kernel in sigsuspend. A busy-waiting program would show state R and close to 100 %CPU. The alarm is not polling the clock; it is waiting for the kernel to wake it.

Signals and handlers

The kernel status file of a waiting alarm contains the following lines.

```
State:  S (sleeping)
VmSize:     2700 kB
VmRSS:      1076 kB
Threads:    1
SigPnd: 0000000000000000
SigBlk: 0000000000000000
SigCgt: 0000000000006002
```

SigCgt is the set of signals for which the process has installed a handler. The value 0x6002 has bits set for signals 2, 14 and 15, that is SIGINT, SIGALRM and SIGTERM, exactly the three that install_handlers() registers. SigBlk is zero because the signals are blocked only for the instant between the flag test and the sleep, and the mask is restored while the process sleeps inside sigsuspend. SigPnd is zero because no signal is waiting to be delivered.

Signals as a control channel

The cancel command ends a waiting alarm by sending SIGTERM with kill(). The log of Test 5 shows the result: the alarm process wrote terminated by signal before expiry and exited, and the cancel command recorded the request. The same mechanism responds to Ctrl+C and to kill from any shell.

Synchronisation: flock() and rename()

Test 8 is the demonstration. Ten processes tried to add a record at the same instant. The lock serialised them, so each saw the previous process's record when computing the next id, and the atomic rename ensured that no reader saw a partial file. The test would fail with duplicate ids or missing records if the locking were removed.

Memory

The virtual size of a waiting alarm is 2700 kB and its resident size about 1 MB. The code pages are shared between alarm processes because they were created by fork() from the same program image. Ten alarms therefore cost far less than ten times the resident size shown for one.

Process life cycle and error handling

Test 11 shows the defence against inconsistent state: when the process behind a PENDING record no longer exists, list reports STALE instead of pretending that the alarm is still armed, and cancel and clear can then be used to tidy the record.